

SERVICIOS EN LA NUBE

BIENVENIDA



Bienvenido(a) a la asignatura *Servicios en la Nube*, con la cual aprenderás sobre los servicios que ofrece Azure, desde bases de datos SQL y NoSQL, hasta buscadores, servicios de seguridad y protección de datos, entre otros.

Los servicios en la nube son infraestructuras, plataformas o sistemas de software que alojan los proveedores externos y que se ponen a disposición de los usuarios a través de Internet, donde estos pueden acceder a distintos servicios incluso si no están instalados en sus ordenadores.



Machiraju, S. & Gaurav, S. (2018) DevOps for Azure Applications: deploy web applications on Azure. Apress.

Rossberg, J. (2019) Agile Project Management with Azure DevOps. Apress..

CASO DISRUPTOR

Te invitamos a revisar el Caso de aplicación:

- **¿Cómo aprovechó Polamer la nube para optimizar la gestión de procesos?**

A través del siguiente video:

Si gustas conocer más acerca de este tipo de casos te invitamos a entrar a la sección de *disruptor* desde tu plataforma de estudio.



1

UNIDAD

DESARROLLO PRÁCTICO DE APLICACIONES AZURE



TEMARIO

1.1 SERVICIOS DE AZURE CLOUD

1.2 BASES DE DATOS CON AZURE CLOUD





INTRODUCCIÓN

La asignatura *Servicios en la Nube* tiene como objetivo conocer los diferentes servicios de la nube que ofrece Azure, Google y DevOps.

En esta primera unidad aprenderás sobre las características de los servicios de la nube de Azure, los cuales tienen múltiples utilidades prácticas.

COMPETENCIAS A DESARROLLAR



El alumno será capaz de comprender y poner en práctica los servicios de la nube que ofrece Azure.



El alumno será capaz de conocer las ventajas del uso de los servicios de nube de Azure en el desarrollo de aplicaciones web.

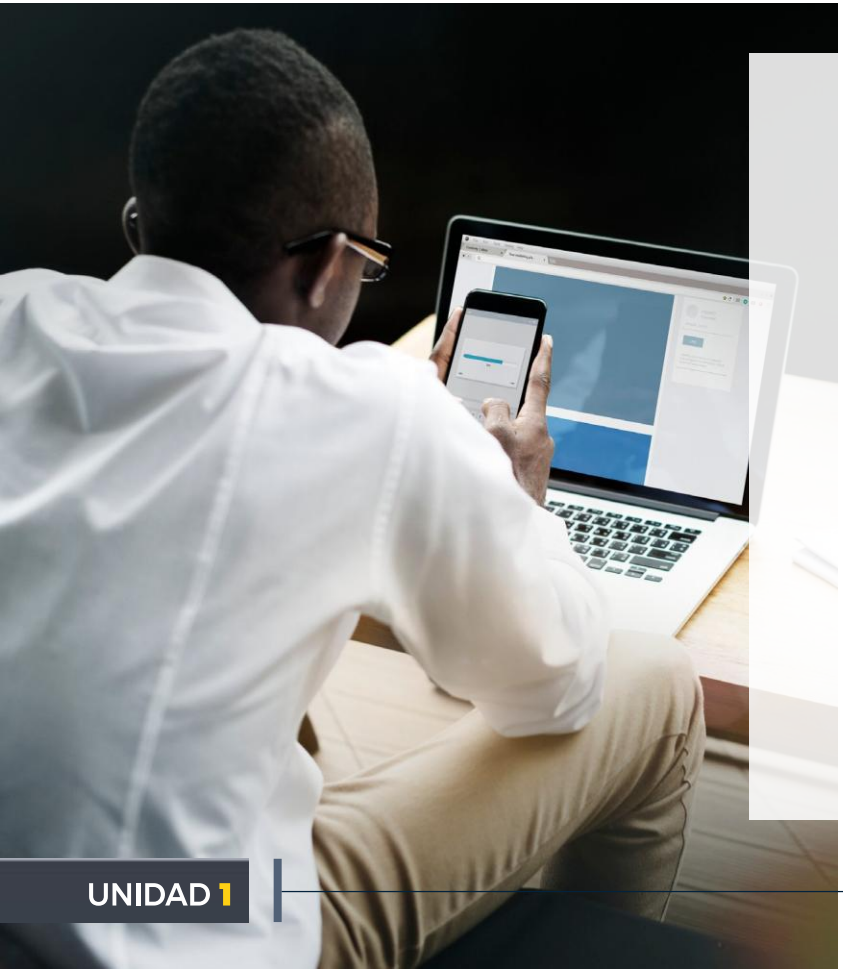
SERVICIOS DE AZURE CLOUD



Solución en la nube. El desarrollo para la nube tiene dos aspectos principales:

- Pensar y diseñar la aplicación para las implicaciones de la nube: costo, latencia, protección de datos, etcétera.

- Conocer las herramientas y servicios disponibles en una plataforma de la nube elegida y aprovecharlos para diseñar soluciones.



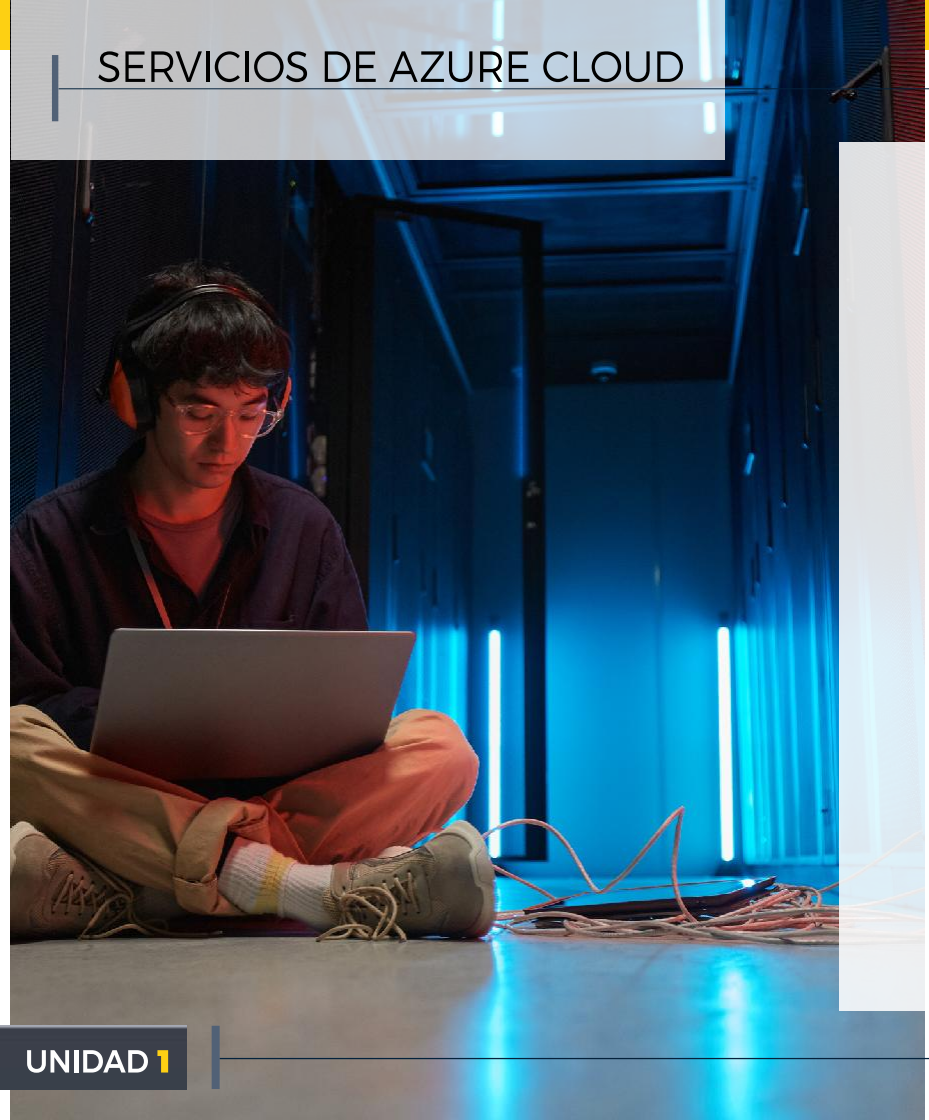
A continuación, te presentamos los **modelos básicos de servicios en la nube** que permanecen constantes entre desarrolladores:

- **IaaS (Infraestructura como Servicio).** Este modelo brinda la flexibilidad y control más cercanos al diseño topológico local de los recursos informáticos. Estos a menudo se abordan en términos de máquinas virtuales, equilibradores de balanza, infraestructura de red, entre otros.

- **PaaS (Plataforma como Servicio).** Este modelo proporciona los recursos informáticos como servicios y API para crear aplicaciones. Estos componentes de servicios en la nube sirven para crear aplicaciones personalizadas.

- **SaaS (Software como Servicio).** Este último modelo ofrece soluciones de software listas para usar, entregadas a través de Internet con las características de la nube.





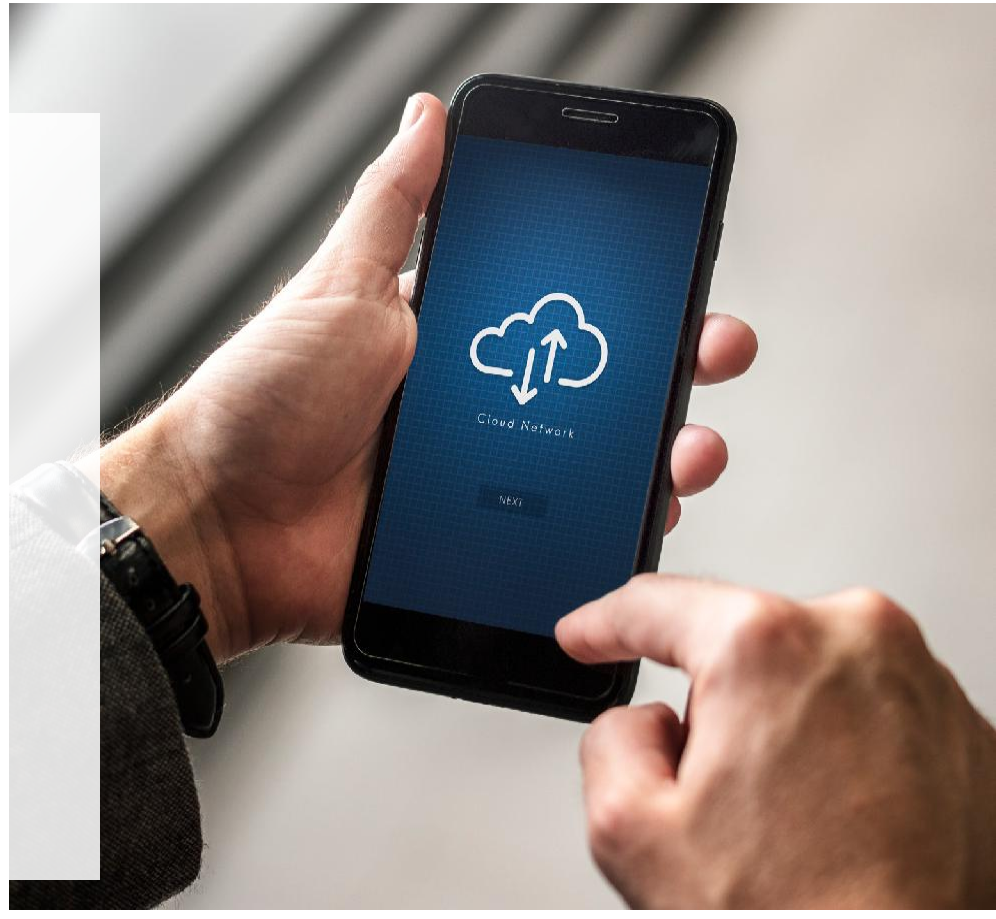
Azure cuenta con dos modelos de implementación de recursos:

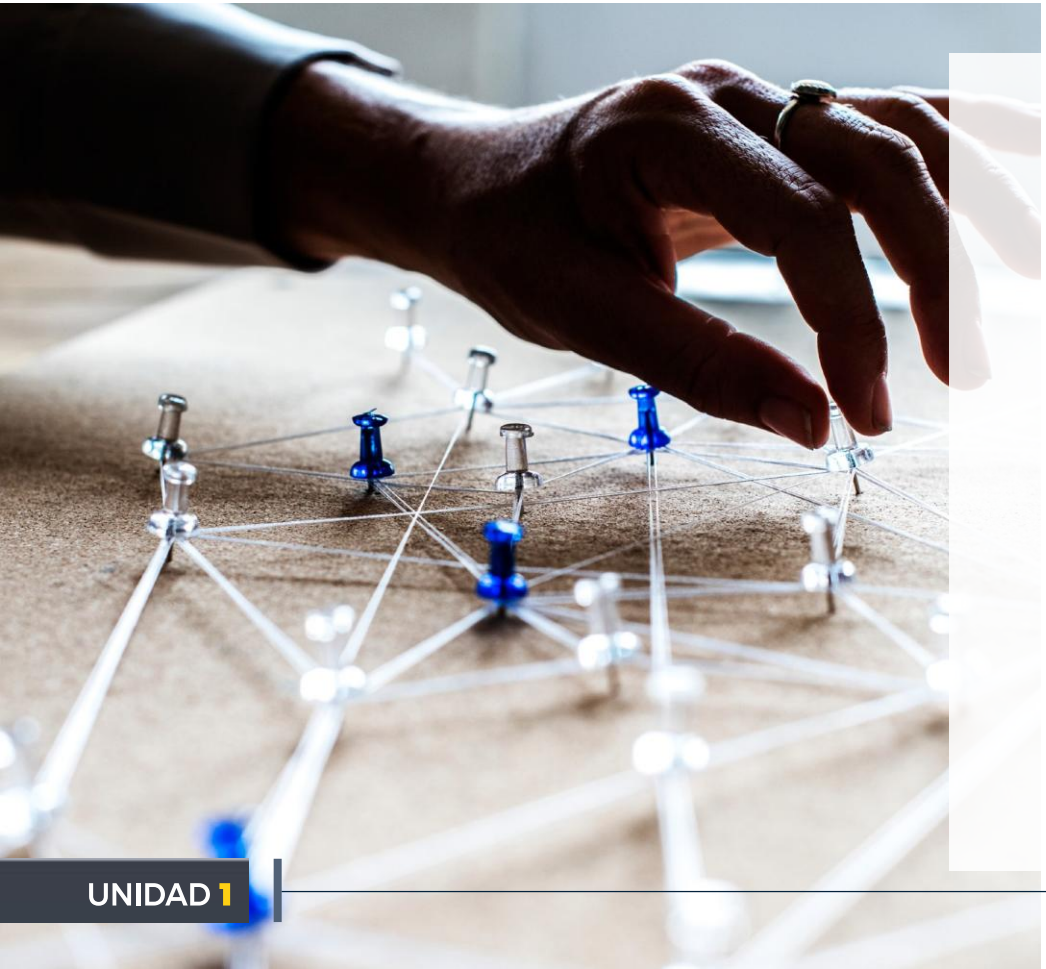
- **El modelo ARM.** Permite administrar los recursos de Azure conocidos como grupos de recursos.

- **El modelo clásico.** Se está volviendo obsoleto. Sus recursos creados a través del portal tradicional a menudo toman el modelo de implementación clásico.

Web Apps vs. Servicio Cloud vs. VMS:

- **Azure Web Apps** es un servicio de plataforma. Es un servidor web como modelo de servicio.
- **Servicio Cloud** es un servicio de plataforma, es decir, de infraestructura.
- **VMS** es una infraestructura pura como modelo de servicio.





Azure SQL Database. *DashDoc* usa la BD SQL para almacenar los datos de la app. Para el desarrollo de la app se usa el enfoque de *Entity Framework* (EF) *Code-First*, con la BD SQL.

La experiencia de desarrollo *Code-First* con Azure SQL Database es análoga a la experiencia con la base de datos local de SQL Server. Para comenzar el desarrollo de la aplicación con Azure SQL Database, necesitamos la cadena de conexión.

Características de la estructura de Azure Blob Storage:

- La cuenta de almacenamiento es el límite de este servicio.
- Dentro de una cuenta de almacenamiento de *blobs* residen los contenedores.
- Todas las *manchas* se almacenan dentro de un contenedor.
- Los contenedores también ayudan a asignar políticas de seguridad.





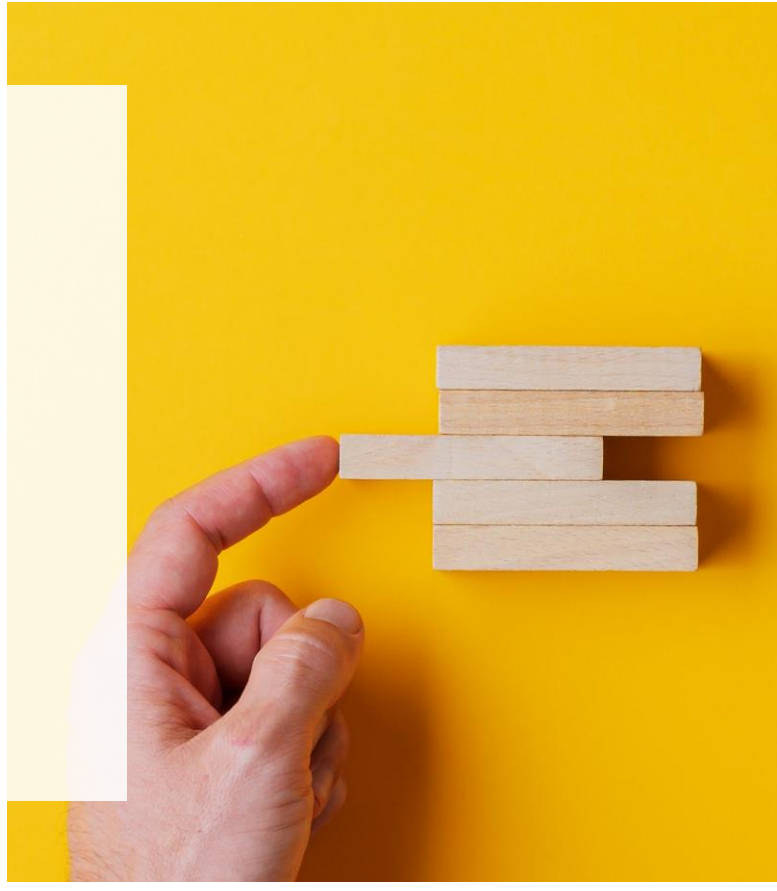
Habilitación de la Supervisión de Aplicaciones mediante Azure

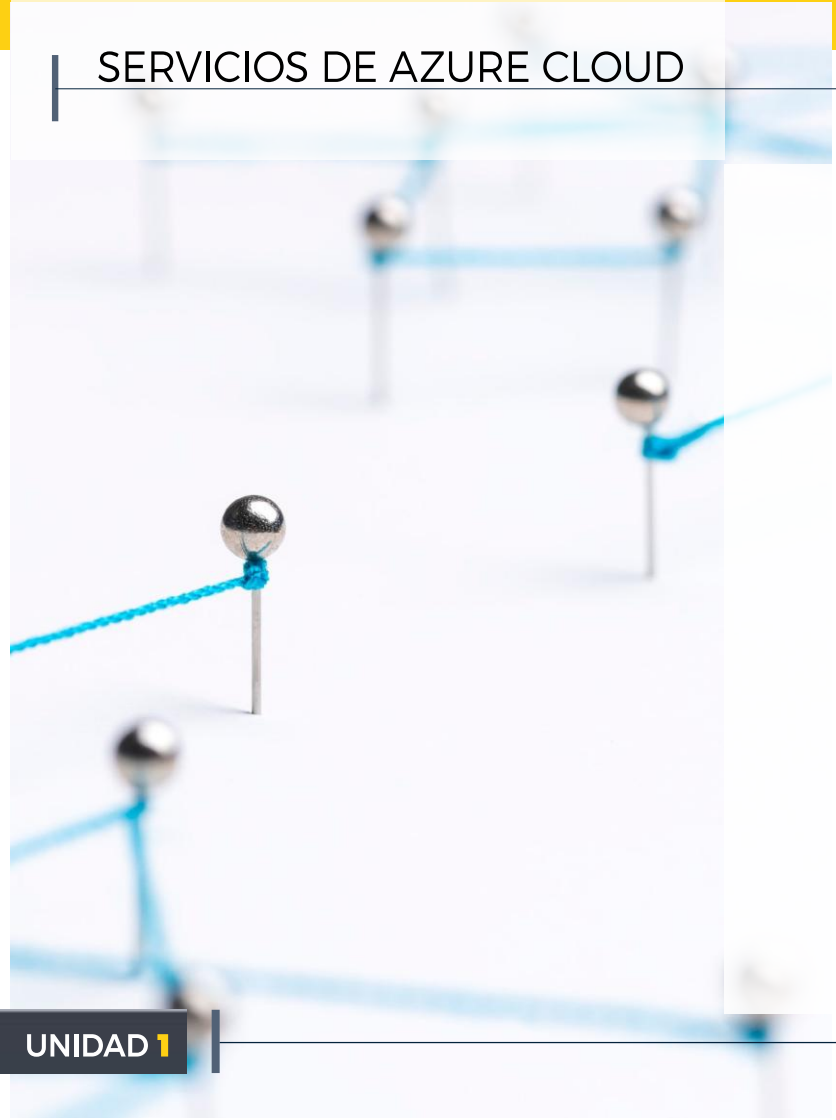
Azure Application Insights es un servicio integral que monitorea y rastrea varias telemetrías de nuestra aplicación. Además, ofrece alertas proactivas que se activarán antes de que ocurra algún posible incidente real.

De la misma manera, podemos ampliar *Application Insights* para cumplir con los requisitos de registro y supervisión de aplicaciones personalizados.

Plantillas de Azure Resource Manager (ARM)

ARM proporciona la capacidad de crear la infraestructura como código en formato JSON. Estas se conocen como plantillas ARM. Además, cuentan con un alto grado de personalización y automatización. También, admiten el aprovisionamiento de recursos incremental y basado en cambios, lo que hace que las plantillas ARM sean una opción sólida para la **automatización de la infraestructura de Azure**.





Manejo de la Configuración del Entorno en las Aplicaciones Web

La creación de diferentes entornos da como resultado diferentes configuraciones para los recursos. Esto se debe a que las cadenas de conexión y las claves de acceso serán diferentes de las que se utilizan en el entorno de desarrollo.

Por esta razón, Azure Web Apps ofrece una *Configuración de la Aplicación Web* para anular la configuración en tiempo de ejecución.

Azure Active Directory and Multitenancy (AAD)

AAD es un servicio de gestión de identidades basado en la nube de Microsoft. Nos permite crear usuarios y grupos para acceder a Microsoft 365 y otras aplicaciones. Los usuarios pueden crearlos en la nube y sincronizarlos con nuestro *AD on Premises* mediante **ADConnect**. Además, esta aplicación nos permite sincronizar uno o varios AD con nuestro Azure AD, teniendo una réplica de nuestros usuarios de AD en la nube.





AAD y la Experiencia de Inicio de Sesión en Azure

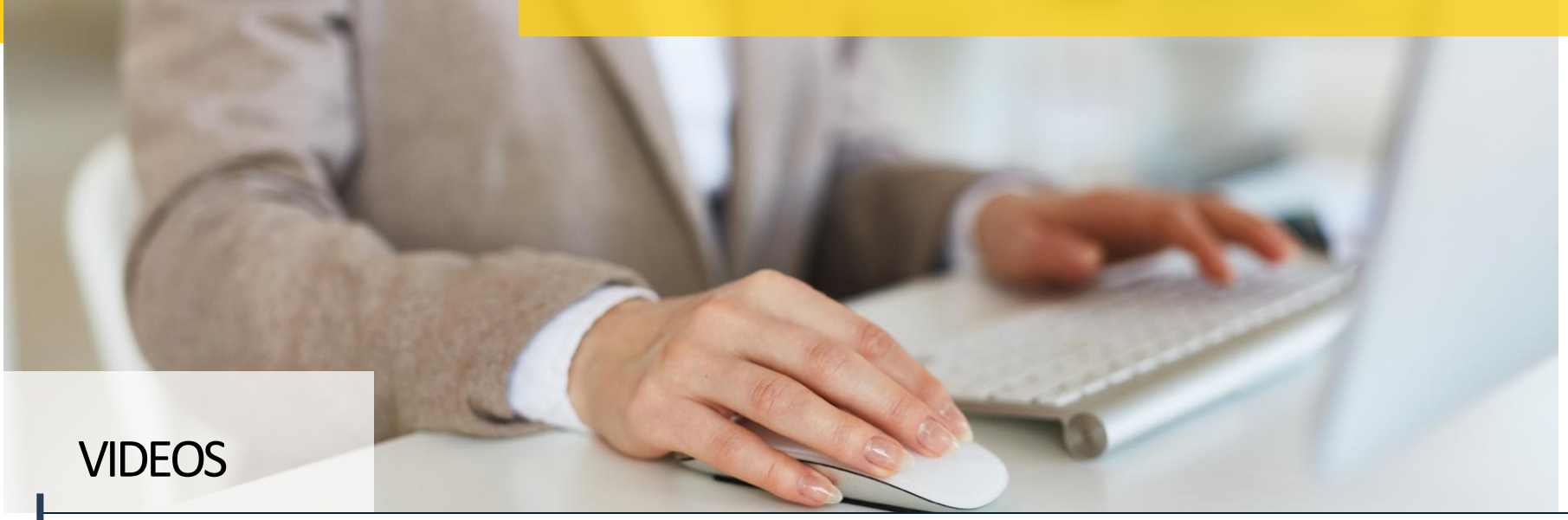
AAD no es una parte o característica inherente a Azure únicamente. Es el servicio de autenticación e identidad común para otros servicios en la nube de Microsoft; de esta manera, Azure solo confía en AAD como su **proveedor de autenticación**.

Habilitar Multitenancy con AAD

AAD se conoce como un *tenant* de Azure, ya que permite su autenticación a una aplicación, y la configura para aceptar las autenticaciones de diferentes AAD, haciendo que una aplicación sea *multitenant.*

Las aplicaciones que sean *multitenant* y que acepten la autenticación desde los AAD obtienen la ventaja del **inicio de sesión único (SSO)** durante los escenarios.





VIDEOS

Te invitamos a ver el siguiente video:

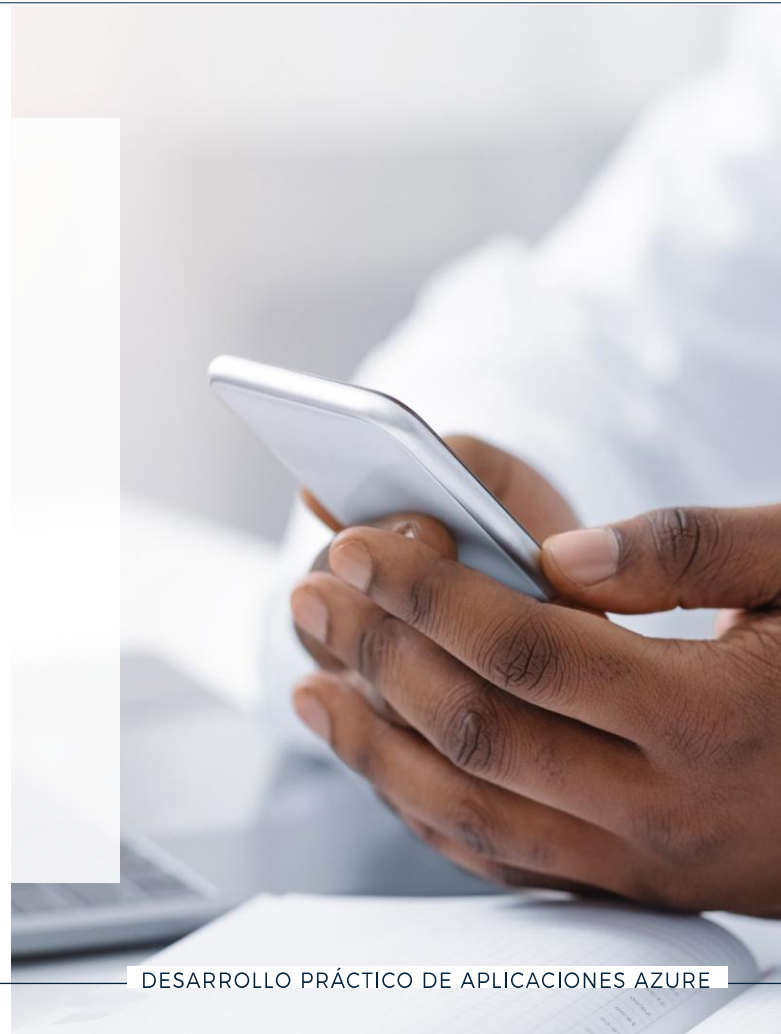
BASES DE DATOS CON AZURE CLOUD



La **persistencia NoSQL** es popular y está ganando un dominio acelerado en la industria. La especialidad fundamental de la persistencia NoSQL es que se pueden escalar sin o con muy pocas restricciones en comparación con las bases de datos de relaciones. Además, también ayuda al desarrollo de aplicaciones modernas con datos crecientes en gran volumen, velocidad y veracidad.

Las **ofertas de servicios** en la nube basados en NoSQL son comunes. La mayoría de los servicios NoSQL basados en la nube ofrecen almacenamiento y escalabilidad ilimitados. Las BD NoSQL también admiten la persistencia y la consulta de datos complejos y no estructurados sin muchas restricciones de esquemas predefinidos.

Azure Table Storage es una BD NoSQL de tipo clave/valor, la cual es muy barata en comparación con otras del mismo tipo.

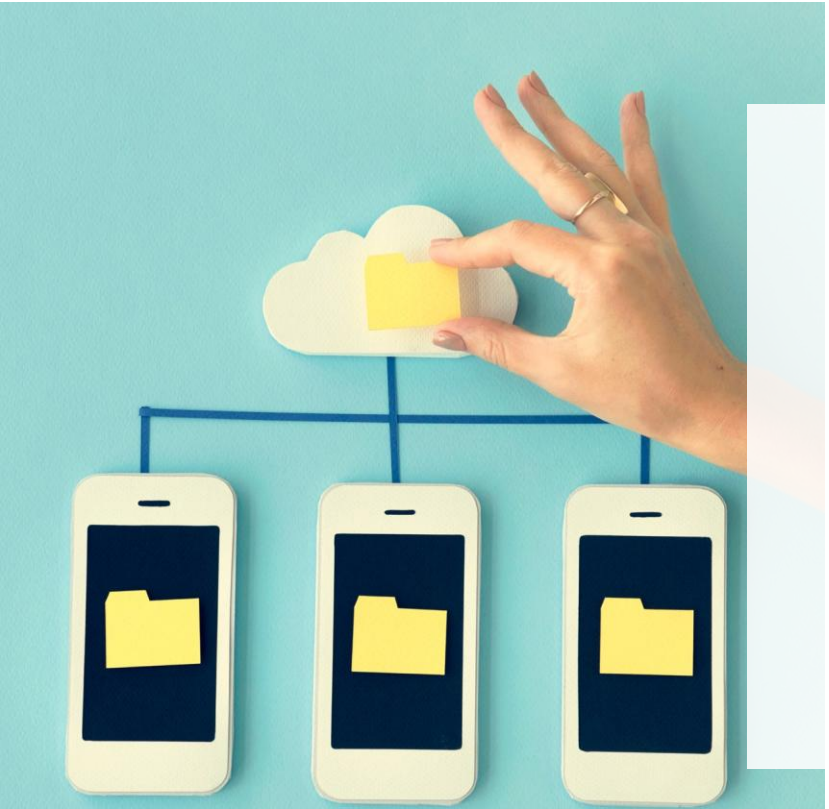


Estructura de almacenamiento de tablas de Azure. La tabla es su estructura base. Una tabla tiene tres atributos incorporados. Por ello, en el momento en que se crea una tabla ya se tienen definidos estos **tres campos**:

- Clave de partición
- Clave de fila
- Marca de tiempo

Además, el almacenamiento de tablas admite **4 tipos de consulta**:

- Puntual
- De rango
- Escaneo de partición
- Escaneo de tabla



Azure Document DB es un servicio de base de datos NoSQL de tipo documento en Azure. Es un servicio basado en JSON, de gran popularidad entre las aplicaciones web y móviles modernas. Además, cuenta con una sintaxis similar a la de SQL para realizar consultas directas, aunque también admite consultas a la API de MongoDB. Por esta razón, Document DB es una opción NoSQL amigable en Azure.

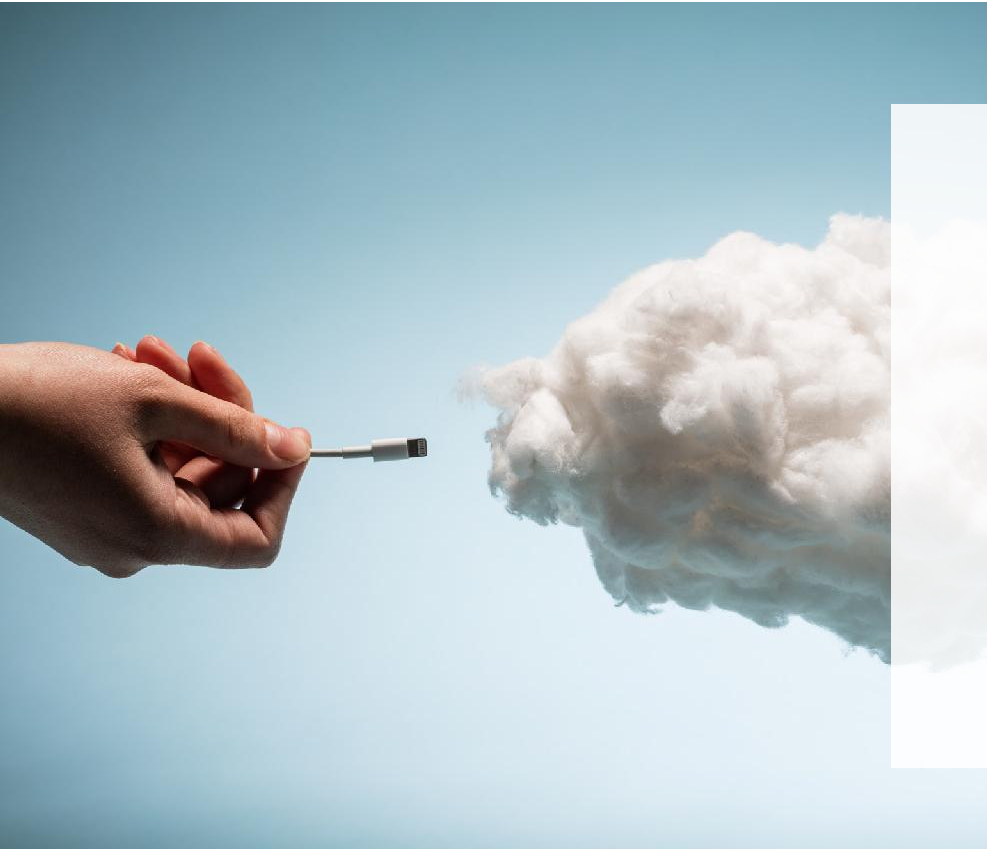
Las bases de datos NoSQL típicas ofrecen **dos niveles de consistencia**:

- Consistencia fuerte y
- Consistencia eventual.

Adicionalmente, Document DB ofrece **dos niveles de consistencia más**:

- Sesión y
- Estanqueidad limitada.





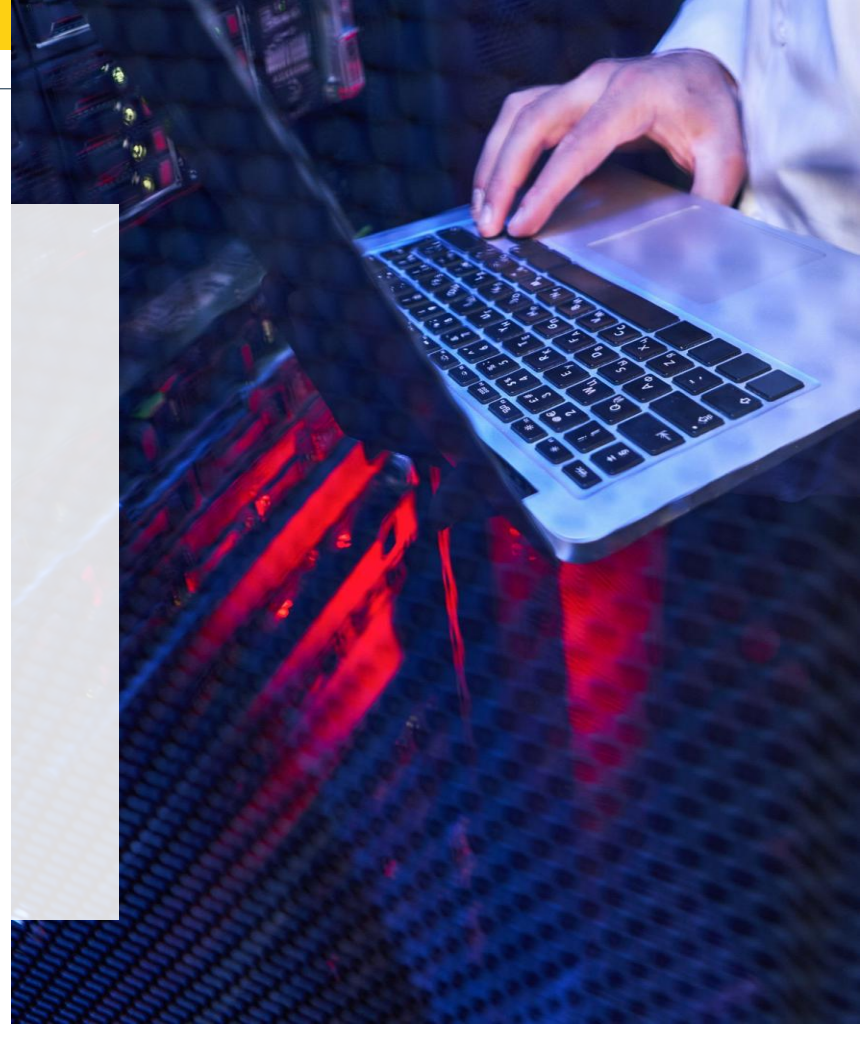
Seguridad y Protección de Datos

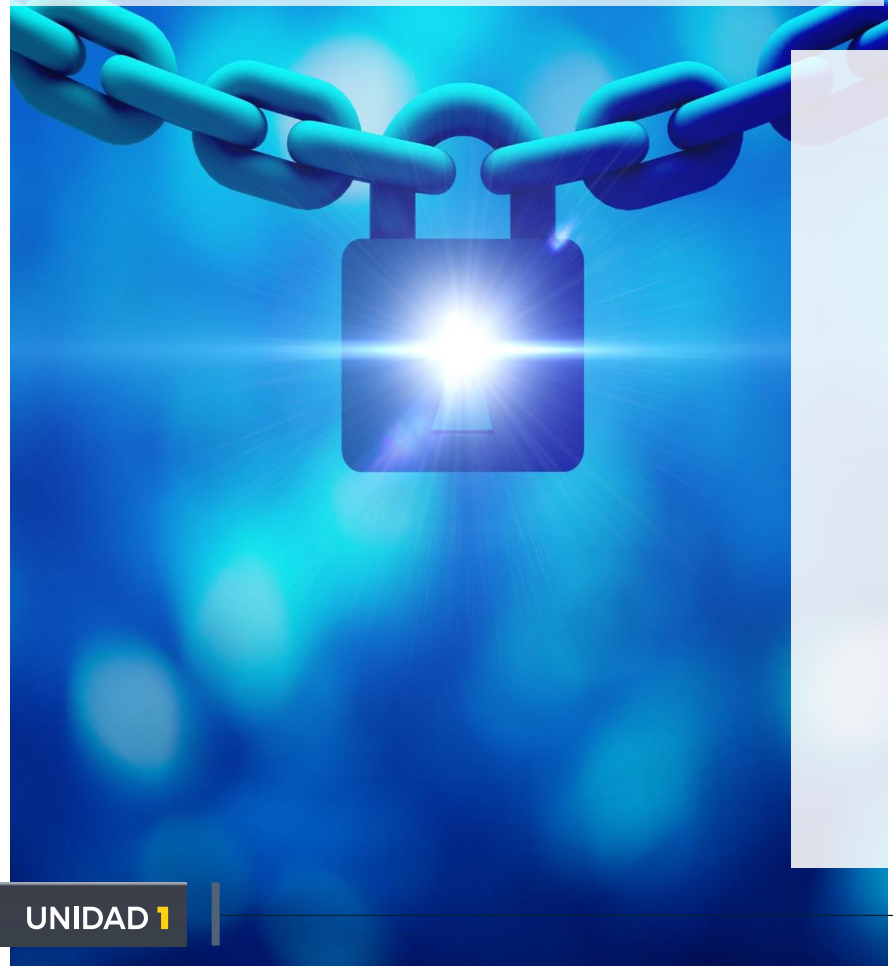
Esta es una de las principales preocupaciones en el desarrollo de la nube. Por lo anterior, Azure SQL Database tiene **seguridad en diferentes capas**.

Además, proporciona un *firewall* para los servidores de bases de datos lógicas.

- **Cifrado Transparente de Datos (TDE).** Permite que los datos, las copias de seguridad y los registros se cifren en reposo. Microsoft administra las claves de cifrado.

- **Enmascaramiento Dinámico de Datos (DDM).** Es una función de SQL Server que enmascara los datos especificados durante la recuperación de datos. Esta no es una opción de cifrado, por lo que los datos en reposo se mantienen como están.

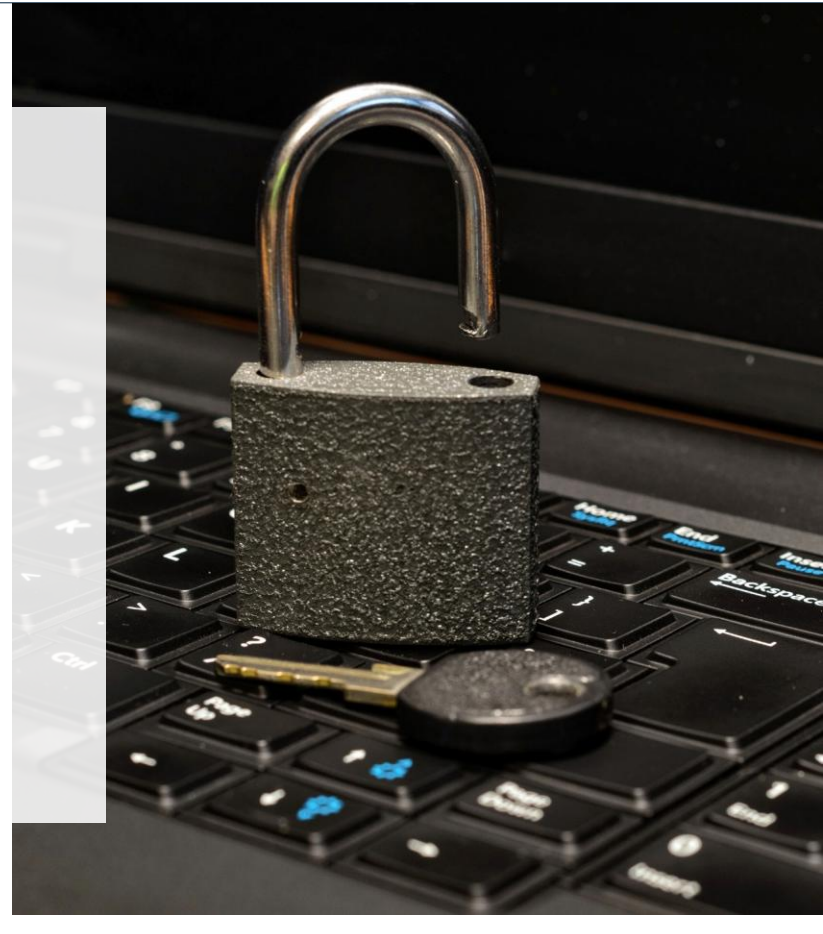


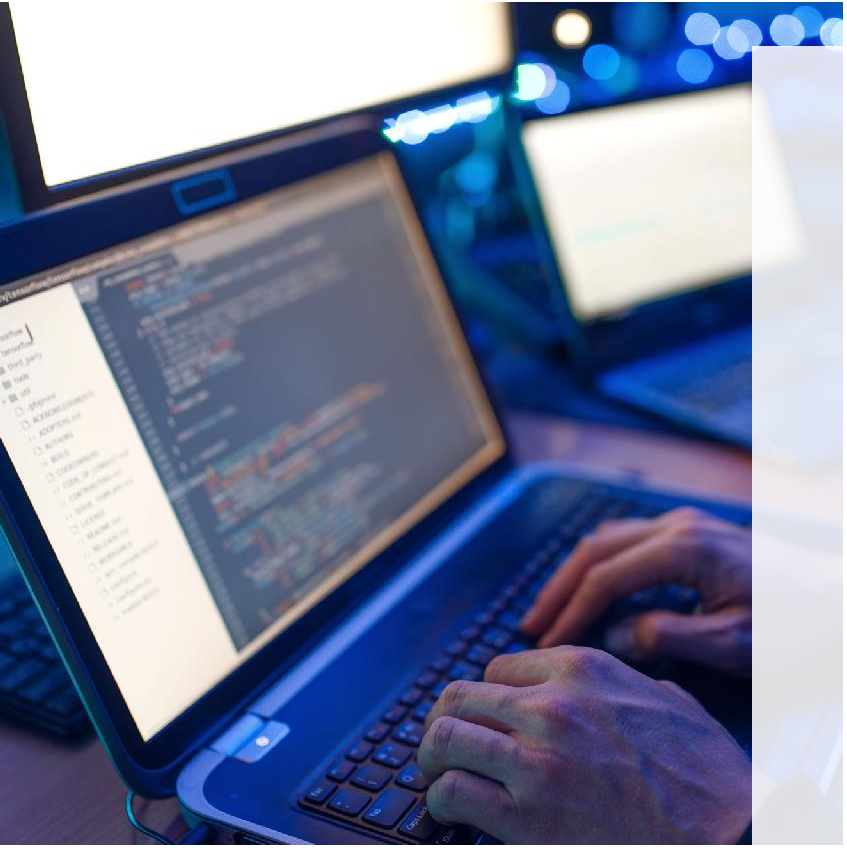


- **Auditoría y detección de amenazas.** Se puede habilitar en el nivel del servidor, por lo que todas las BD heredan la configuración del servidor. Sin embargo, cada base de datos también puede tener habilitada su propia supervisión y detección de amenazas. Azure usa el cifrado **AES 256** para el cifrado de almacenamiento. Cuando se habilita esta opción después de que se crea el archivo, su contenido permanece sin cifrar hasta que los usuarios acceden y guardan los datos.

Azure Key Vault. Es un servicio de administración de claves que no solo administra las claves basadas en software, sino también las basadas en hardware; esto lo logra a través de módulos de seguridad (HSM). Podemos generar claves o importar las existentes en Azure Key Vault. Hay **2 opciones** para manejar esta información:

I. Secretos
II. Claves





Integración de Azure Search

Es un servicio de modelo de búsqueda donde la administración de recursos de búsqueda lo realiza Azure, y los desarrolladores se enfocan en la funcionalidad de búsqueda.

Asimismo, ofrece características avanzadas como soporte para búsqueda multilingüe, descifrado de documentos, soporte de indexación integrado para varias fuentes de datos, sugerencias de búsqueda, resaltado de resultados, exploración de facetas, búsqueda geoespacial y más.

Un servicio de búsqueda funciona con:

- Un buscador de texto completo con almacenamiento de contenidos propios del usuario en un índice de búsqueda.
- Indexación enriquecida, con análisis de texto y enriquecimiento opcional de IA para la extracción y transformación avanzada de contenidos.
- Funciones de consulta completas, incluidas las sencillas, la sintaxis completa de Lucene y la búsqueda por tipo.



A continuación, te presentamos los **componentes importantes de Azure Search** necesarios para comenzar:

- SKU del servicio de búsqueda.
- Índice de búsqueda.
- Indexador de búsqueda.
- Fuente de datos.
- SQL database.



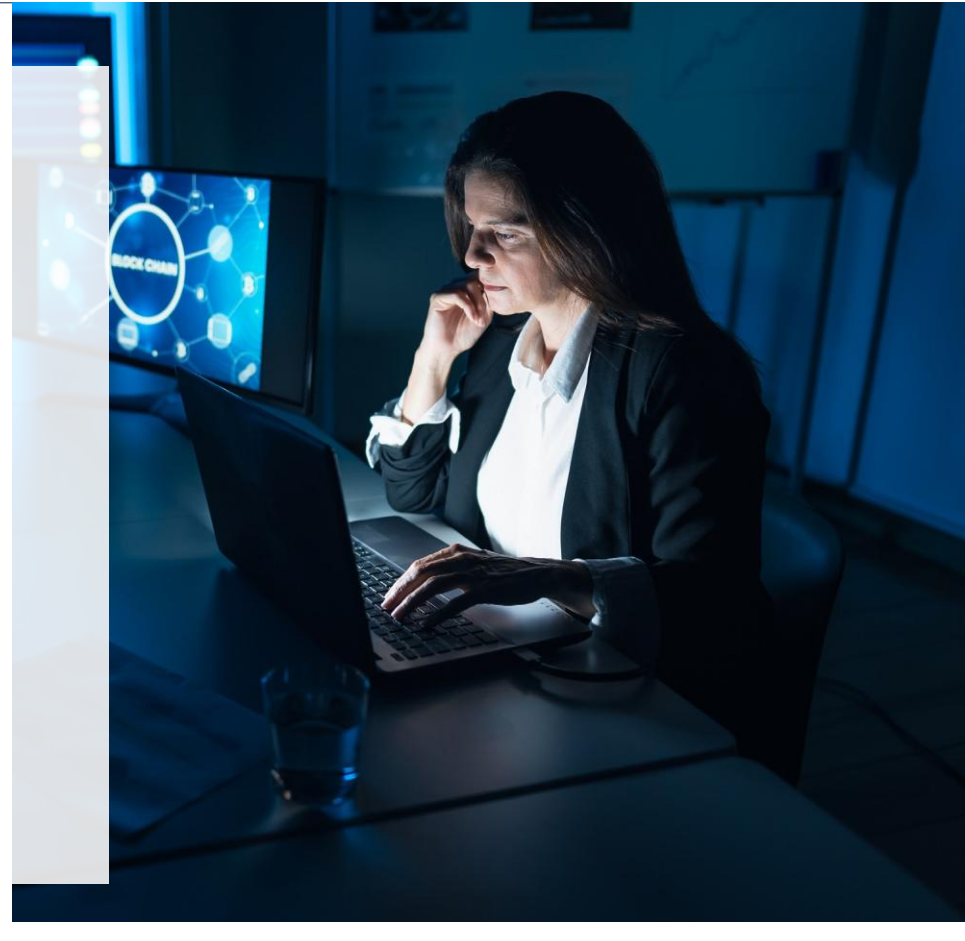


Power BI Embedded

Nos permite tener visualizaciones interactivas ricas en datos integradas en aplicaciones personalizadas, sin la necesidad de licencias de Power BI independientes. Todo el servicio está administrado por Azure, y nos brinda las herramientas de escritorio de Power BI para crear informes.

Power BI Embedded nos permite:

- **Crear informes interactivos fácilmente.** Solo hay que descargar Power BI Desktop y crear desde el escritorio estos informes interactivos.
- **Escalar en la nube de manera eficaz.** Dispondremos de todos los servicios integrados en la plataforma de Azure.
- **Desarrollarse en un entorno abierto.** Podremos utilizar sus herramientas favoritas, dentro de un entorno familiar.





Power BI cuenta con posibilidades de configuración, pero son limitadas. Los principales **retos a los que se enfrentan las empresas al trabajar con Power BI son:**

- Gestionar la publicación de los informes.
- Conceder accesos a usuarios que cumplen roles diferentes.
- Asegurar la seguridad de los datos, informes y contenidos.
- Acompañar y capacitar a usuarios avanzados.



LECTURAS PARA REFORZAR LA UNIDAD

- Luijbregts, Michael Crump & Barry. (2019). *Azure Developer Guide EBook Es.*
- Michael Collier & Robin Shahan. (2016). *Fundamentals of Azure.* Second Edition. Redmond, Washington: Microsoft Press.
- Sánchez, Óscar Moreno. (2016). *Servicios en la nube con Microsoft Azure.* Madrid.

FORO 1

De acuerdo al caso de aplicación ¿Cómo aprovechó **Polamer la nube para optimizar la gestión de procesos?,** el cual puedes ver mediante el siguiente enlace:

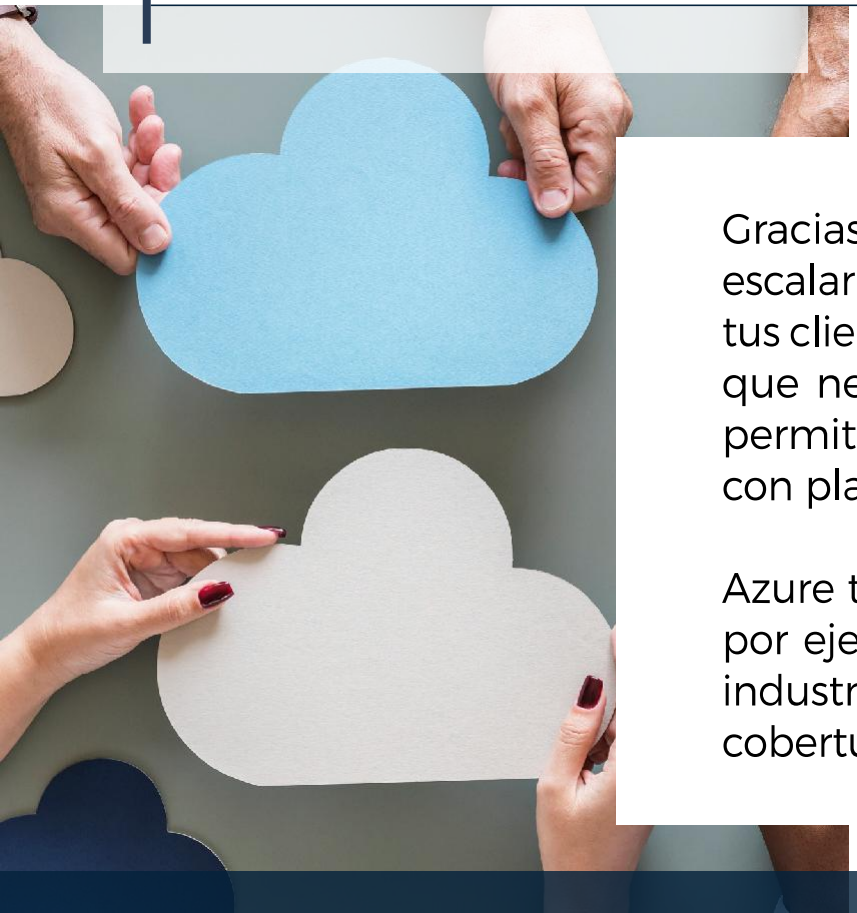


- ¿consideras que realmente la solución implementada ayudó a optimizar la gestión de procesos de la empresa?

- ¿qué solución le hubieras tu propuesto a la empresa y de qué forma se hubiera implementado?

Presiona el botón para participar en el foro.

CONCLUSIÓN



Gracias al hospedaje de aplicaciones en Azure, se puede escalar fácilmente a medida que cambian las necesidades de tus clientes. De la misma manera, proporciona la confiabilidad que necesitas para aplicaciones de alta disponibilidad y te permite administrarlas fácilmente mediante programación, con plantillas y API específicas del servicio.

Azure también nos brinda una serie de garantías adicionales, por ejemplo: bajo precio, alta disponibilidad, liderazgo en la industria en certificaciones de seguridad y, de hecho, tiene la cobertura de cumplimiento más alta de cualquier nube.

¡FELICIDADES!



Acabas de concluir la primera unidad de tu curso *Servicios en la Nube*. Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente. Para ello, debes regresar a la pantalla principal y dar clic en *Presentar examen*.



2

UNIDAD

PRO DEVOPS CON GOOGLE CLOUD PLATFORM

TEMARIO

2.1 | INTEGRACIÓN Y ENTREGA CONTINUAS

2.2 | ARQUITECTURA DE MICROSERVICIOS





INTRODUCCIÓN

En esta segunda unidad aprenderás a describir las capacidades de DevOps de Google Cloud Platform que se pueden usar para adoptar procesos de CI/CD al automatizar el ciclo de vida de las aplicaciones.

De la misma manera, conocerás los beneficios de usar Google Cloud Platform para la gestión de la configuración, así como las herramientas proporcionadas para ello.

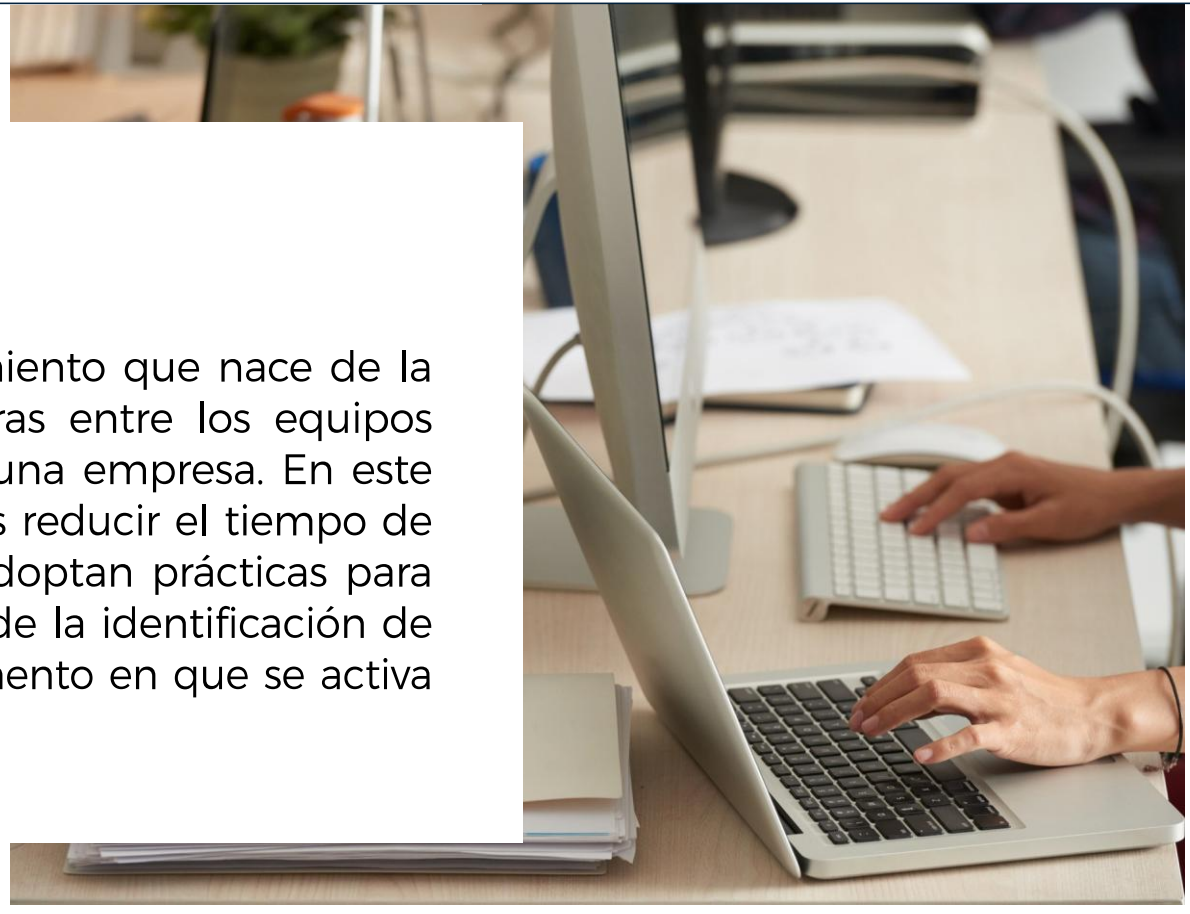
COMPETENCIAS A DESARROLLAR



El alumno será capaz de comprender la ingeniería DevOps utilizando el escenario Google Cloud Platform.



El alumno conocerá los beneficios de utilizar la integración y entrega continuas en sus proyectos de desarrollo de software.

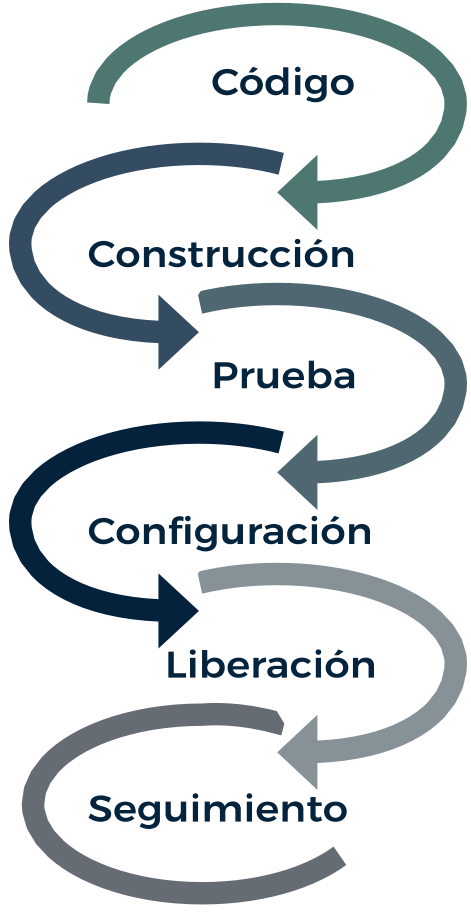


¿Qué es DevOps?

Se utiliza para definir un movimiento que nace de la necesidad de reducir las barreras entre los equipos de desarrollo y operaciones de una empresa. En este sentido, el objetivo de DevOps es reducir el tiempo de comercialización. Para ello, se adoptan prácticas para reducir el tiempo necesario, desde la identificación de un nuevo requisito hasta el momento en que se activa para los clientes.

Cadena DevOps

Esta cadena de herramientas se utiliza para definir cada paso del proceso de producción. Cada fase puede ser gestionada por un equipo diferente. Por esta razón, es importante la coordinación y comunicación sólidas y claras. Las fases de la cadena son:

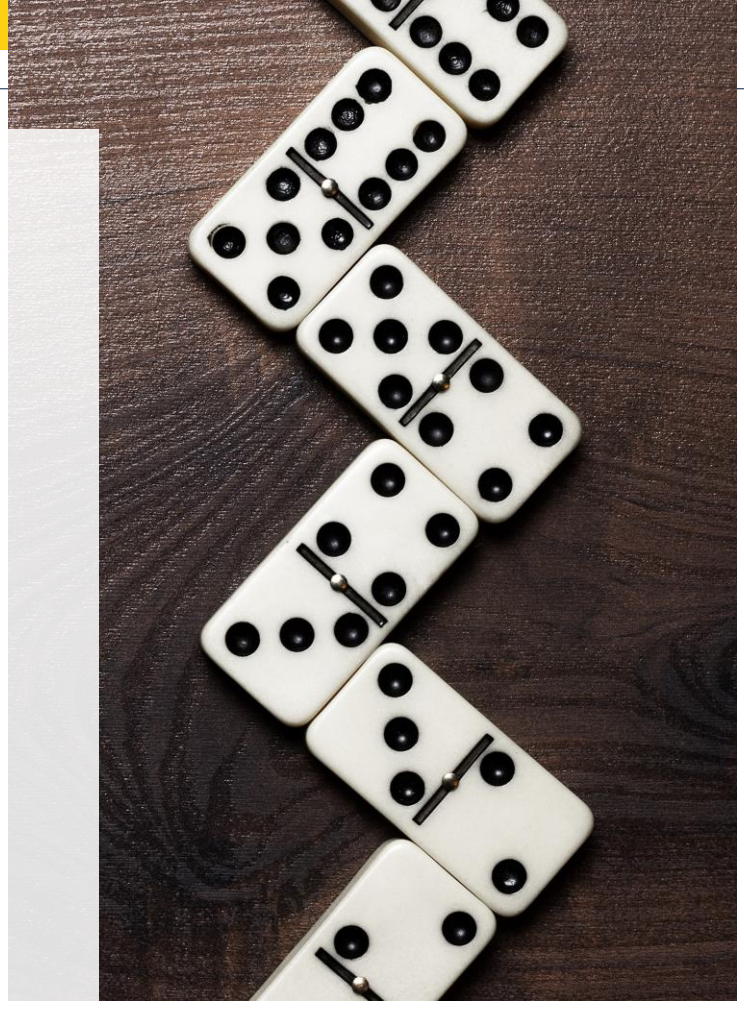


Integración Continua

La CI representa el primero de los cambios en el ciclo del desarrollo, los cuales se resumen en:

- Definir la prueba unitaria.
- Definir una política de sucursal.
- Disponer de un sistema de integración continua

Tener la versión QA es importante tanto para corregir errores como para iniciar la fase de monitoreo. Este último es vital en DevOps, ya que permite reducir y prevenir errores en el sistema.





Google Cloud

Es un conjunto de infraestructuras y servicios que Google usa internamente. Está disponible para cualquier empresa, ya que se adapta a muchos procesos comerciales.

Adicionalmente, nos proporciona todas las herramientas que necesitamos para diseñar, probar y ejecutar aplicaciones desde Gcloud, con una escalabilidad y seguridad muy superiores a las de cualquier otra herramienta.

Computación en la Nube

NIST: “La computación en la nube es un modelo para permitir el acceso a recursos informáticos configurables que pueden aprovisionar y liberar rápidamente con un mínimo de esfuerzo de administración o interacción del proveedor de servicios”.

Este modelo se compone de cinco características esenciales, tres modelos de servicio y cuatro modelos de implementación.





La **nube** es esencialmente un recurso compartido dentro de una red. Cuenta con las **siguientes características:**

- Autoservicio y bajo demanda.
- Acceso a la red.
- Agrupación de recursos.
- Respuesta al cambio más rápido.
- Servicio medible.

Otra característica importante de la computación en la nube son sus modelos de servicio y despliegue.

Modelos de Implementación

En la computación en la nube podemos identificar **4 modelos**, los cuales son:



Privado

Está destinado solamente para su uso interno.



Comunidad

Está destinado para usarse en una comunidad específica.



Público

Este modelo puede ser usado por cualquier persona.



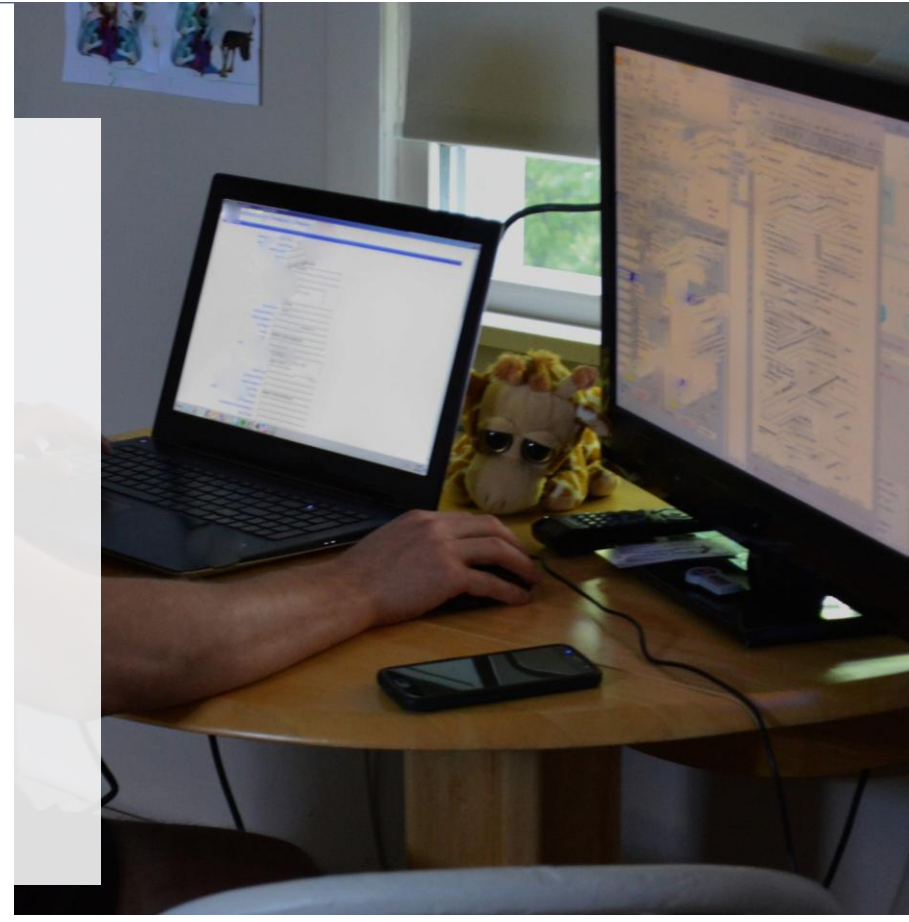
Híbrido

Combina diferentes modelos mencionados anteriormente.

La integración continua (CI) es una práctica de desarrollo que requiere que el desarrollador integre el código en un repositorio compartido central. Cada vez que el desarrollador confirma el código, este se integra con otro código y se verifica mediante la ejecución de una prueba.

¿Qué es un escenario CI?

En un escenario de CI, una compilación va más allá que compilar software, ya que se compone de todas las operaciones necesarias para lanzarlo.



Entrega Continua

La CD es una práctica que se utiliza para lanzar software en un ciclo corto. De esta manera, con cada compilación creamos una nueva compilación de todo el software.

Lo anterior permite que el equipo de control de calidad comience a realizar pruebas de inmediato, con un número restringido de funciones, y brinde retroalimentación inmediata al equipo de desarrollo.





Diferencia entre CI y CD

Mientras que la CI se concentra en integrar el software con cada compromiso, la CD amplía este proceso, ya que agrega otra capa después de que se completan la integración y las pruebas del software. En este sentido, la CD crea el software y lo prepara para su posible lanzamiento.

Mientras la CI pone un gran énfasis en la **fase de prueba**, la CD lo pone en la **creación de software**.

Para tener una buena CD, debemos contar con:

- Buenas estrategias de ramificación.
- Una política sólida de pruebas unitarias.
- Una fase de prueba automática.
- Promoción automática de código.

Todas las prácticas anteriores están estrictamente conectadas y ayudan a producir una CD buena y fuerte. Algunas contribuyen, por su parte, a una CI sólida.



Para construir un buen sistema para CI y CD, se crea una **canalización**. Con ella se definen los pasos necesarios para construir el software y, finalmente, lanzarlo en producción.

Cuando se crea software, se identifican diferentes etapas. Cada etapa es responsable de una validación específica. En este sentido, la tubería básica de esta canalización se compone de **tres etapas**:

DESARROLLO

PUESTA EN ESCENA

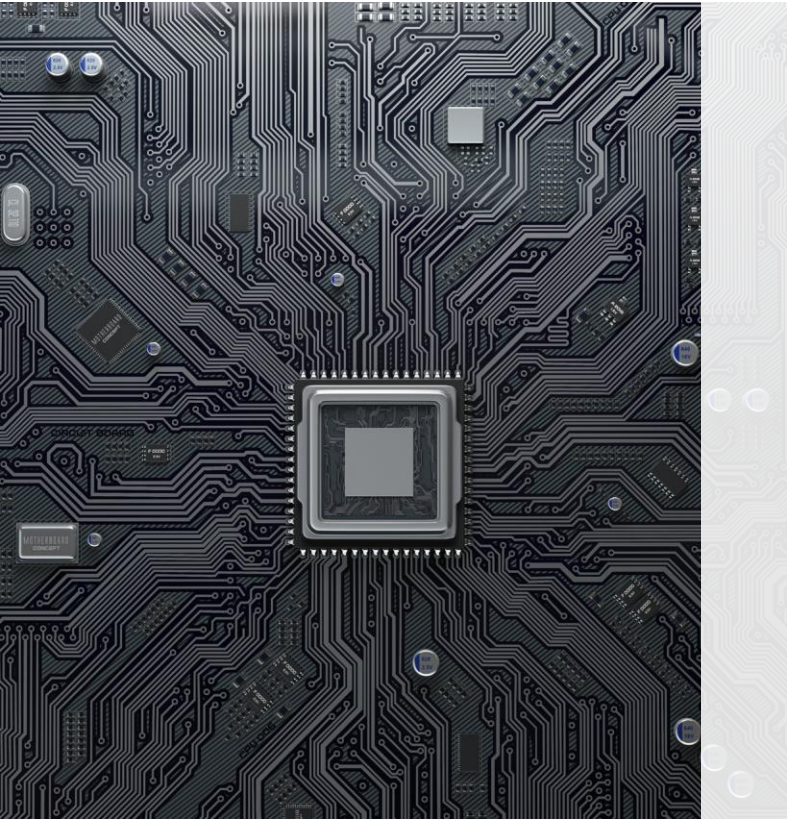
PRODUCCIÓN

Integración Continua de BD

La integración continua de la base de datos es el proceso de liberar y reconstruir toda la base de datos y completarla con nuevos datos, cada vez que lanzamos el software.

Es importante establecer una integración continua de la base de datos, la cual es comúnmente conocida como **migración de la base de datos**.





Google Compute Engine

Es el componente IaaS que se ofrece en GCP. **Se compone de tres elementos básicos**:

- Máquina virtual
- Componente de red
- Disco persistente

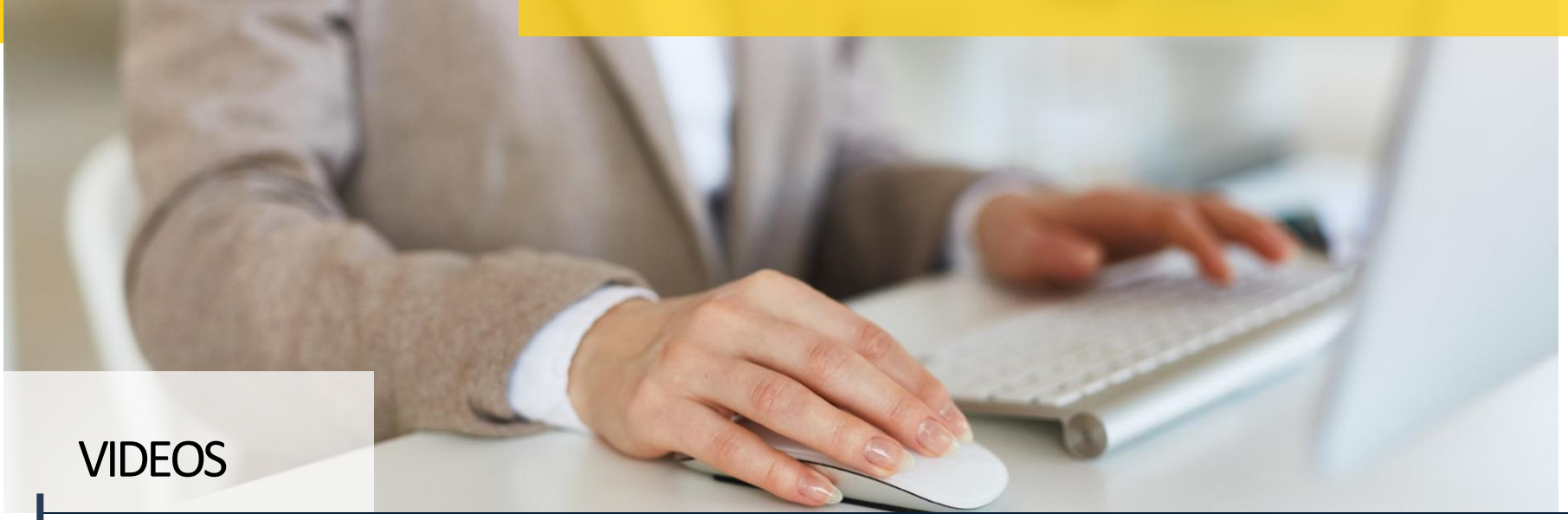
Con Google Compute Engine se puede crear un flujo de trabajo para escalar desde una sola instancia a una instancia distribuida. Cuando creamos un Google Compute Engine, podemos elegir cualquier configuración en términos de CPU, memoria y espacio.

Jenkins

Es un software de código abierto escrito en Java para ayudar a automatizar todos los procesos no humanos que ocurren durante el desarrollo de software. Jenkins funciona con casi cualquier tipo de repositorio, como CVS, Git, Subversion, Mercurial y otros.

Es posible automatizar todos los procesos relacionados con CI/CD de forma muy sencilla. El concepto más importante de Jenkins es el complemento.

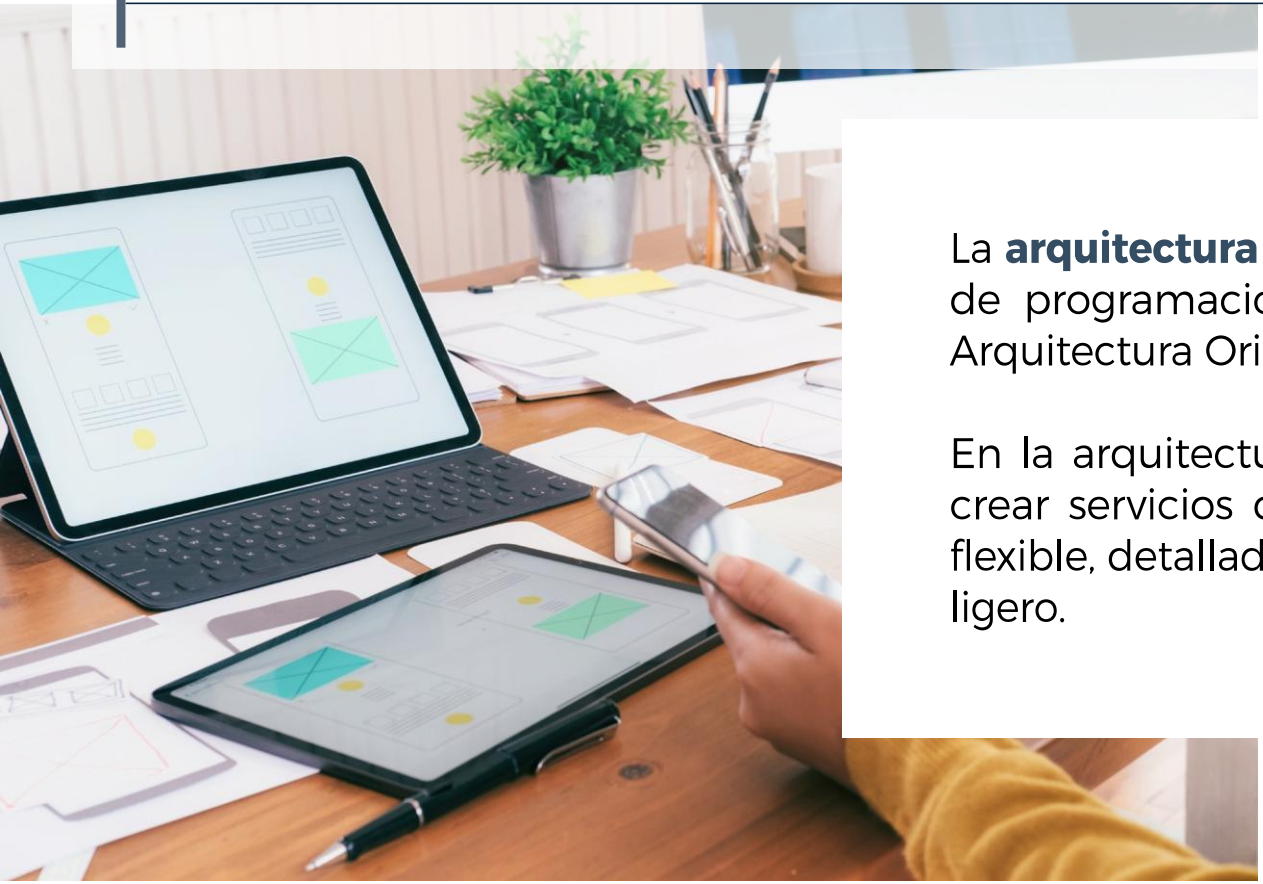




VIDEOS

Te invitamos a ver el siguiente video:

ARQUITECTURA DE MICROSERVICIOS



La **arquitectura de microservicios** es una técnica de programación derivada directamente de la Arquitectura Orientada a Servicios (SOA).

En la arquitectura de microservicios, se pueden crear servicios que están acoplados de manera flexible, detallados y conectados con un protocolo ligero.

A continuación, se muestran algunos **principios asociados con la arquitectura de microservicios:**

Los microservicios están diseñados para grandes sistemas

Un microservicio nace de la necesidad de escalar un gran sistema, pero la forma en que se define un gran sistema es relativa.

Los microservicios están orientados a objetivos

Cuando se diseña una arquitectura de microservicios no hay que seguir reglas establecidas, pero se pueden hacer soluciones diseñadas para resolver problemas específicos.

Los microservicios están diseñados para ser reemplazados

Se diseña un servicio para que esté débilmente acoplado a otros, lo que permite cambiar un servicio por otro, sin crear problemas para el sistema.

Implementación

Cuando implementamos una arquitectura de microservicios, tomamos la decisión de tener un pequeño servicio diseñado para responder a la necesidad empresarial de comunicarse con una cantidad mínima de datos. Esto requiere un cambio cultural para adaptarse a eso. Este cambio está relacionado con la filosofía de Unix, la cual pregona: “hacer una cosa y hacerla bien”. Con base en esto, podemos definir la filosofía y los principios arquitectónicos de los microservicios.





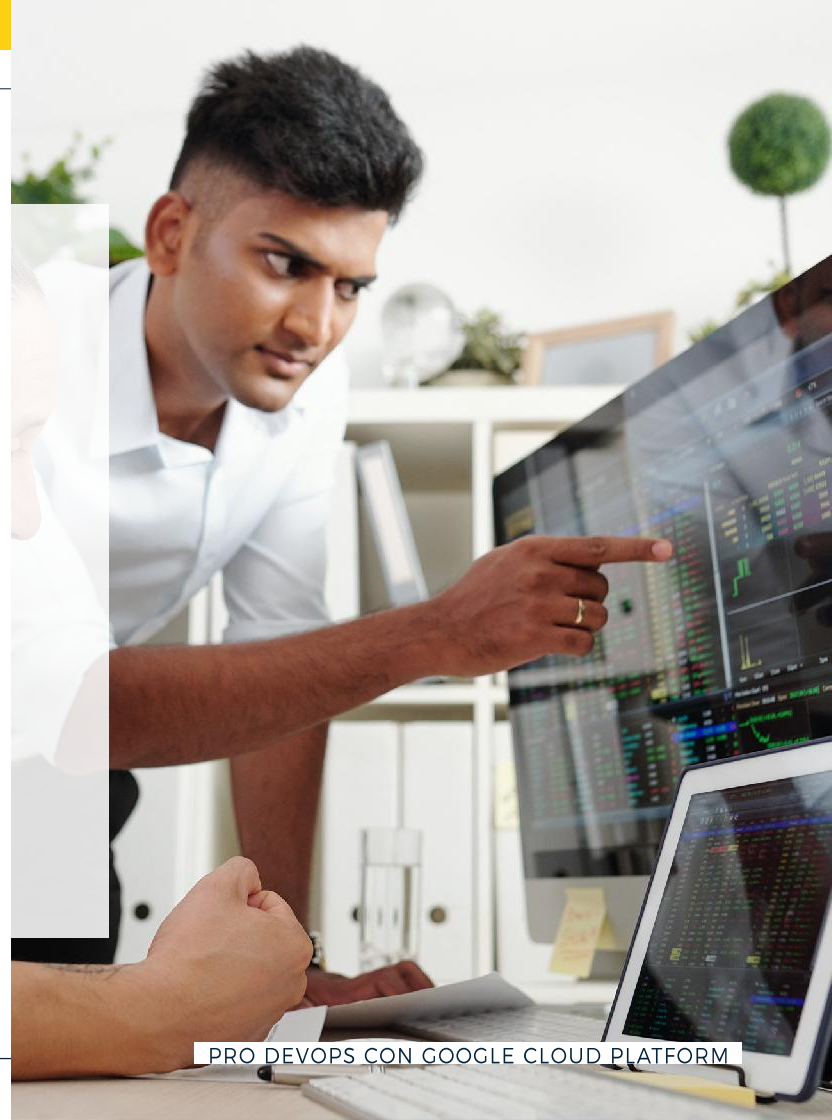
Monitoreo en GCP

Tener un buen sistema de monitoreo es crucial para obtener comentarios precisos sobre la infraestructura y el software que hemos creado, así como para supervisar el acceso al mismo.

Un software de monitoreo garantiza la calidad del servicio, según los términos de SLA. Se compone de métricas diseñadas en el software, que son representadas gráficamente. Estas se recopilan y analizan para proporcionar una imagen del software y cómo funciona.

Los requisitos clave de un sistema de monitoreo son:

- **Métricas.** Hay que definirlas para recopilar datos sobre el estado actual del sistema.
- **Alertas.** Definir un sistema que brinde alertas cuando las métricas indiquen un valor incorrecto.
- **Monitoreo.** Usar un software para recopilar y mostrar el estado real del sistema.
- **Retroalimentación.** Es necesario que esta sea proactiva, en caso de errores en el sistema.



Cuando tenemos que construir un sistema de monitoreo, hay conocer ciertos componentes clave. La primera suposición que debemos hacer es que no hay una definición clara de monitoreo.

Algunos **factores clave para hacer un sistema de monitoreo** son los siguientes:



La **importancia del monitoreo** es multifactorial, ya que no solo sirve para garantizar la estabilidad del sistema en sí. Por ejemplo, **un sistema de monitoreo se puede utilizar para lo siguiente**:



Monitoreo de Caja Blanca

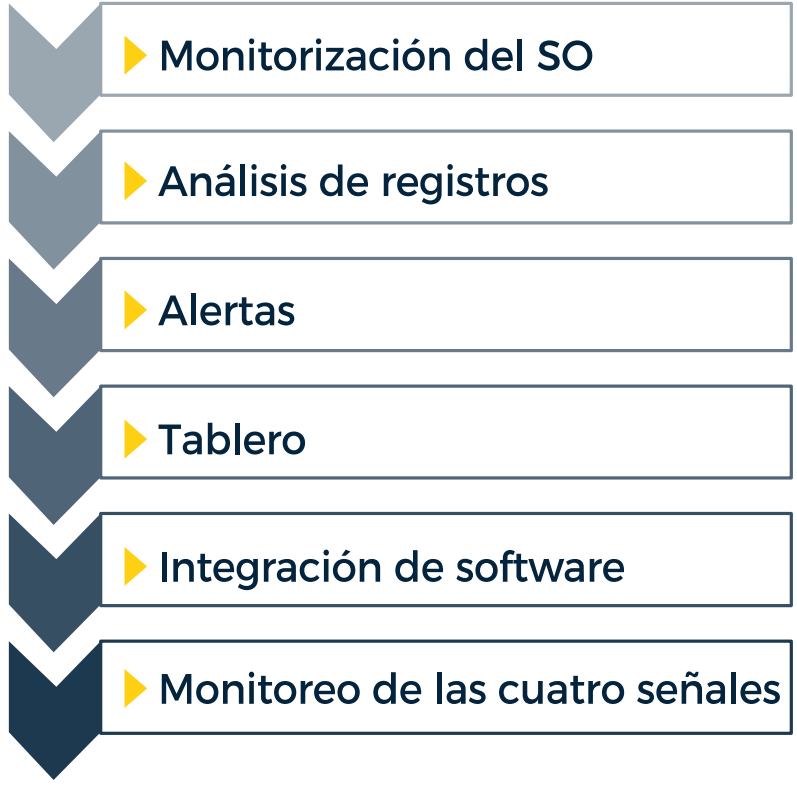
El monitoreo de caja blanca se puede realizar en todo el software y los sistemas. Además, se puede configurar para revelar el estado interno del software. De la misma manera, se utiliza el software para verificar el sistema operativo.

Existe una regla de oro que se aplica a los sistemas de monitorización. Esta define **cuatro señales doradas** que se pueden usar para obtener una imagen del sistema. Estas señales son:



Es tiempo de definir los requisitos y el software necesarios para construir **el sistema de monitoreo**.

Supongamos que queremos construir nuestro sistema de monitoreo desde cero. Para ello, debemos considerar los siguientes puntos, los cuales son esencialmente los principales requisitos para construir el sistema:



Creación y Administración de Infraestructura en GCP

Uno de los puntos críticos de DevOps es la infraestructura. En este sentido, una vez que esta se diseña e implementa en la nube, se desea que continuamente los resultados sean los mismos.

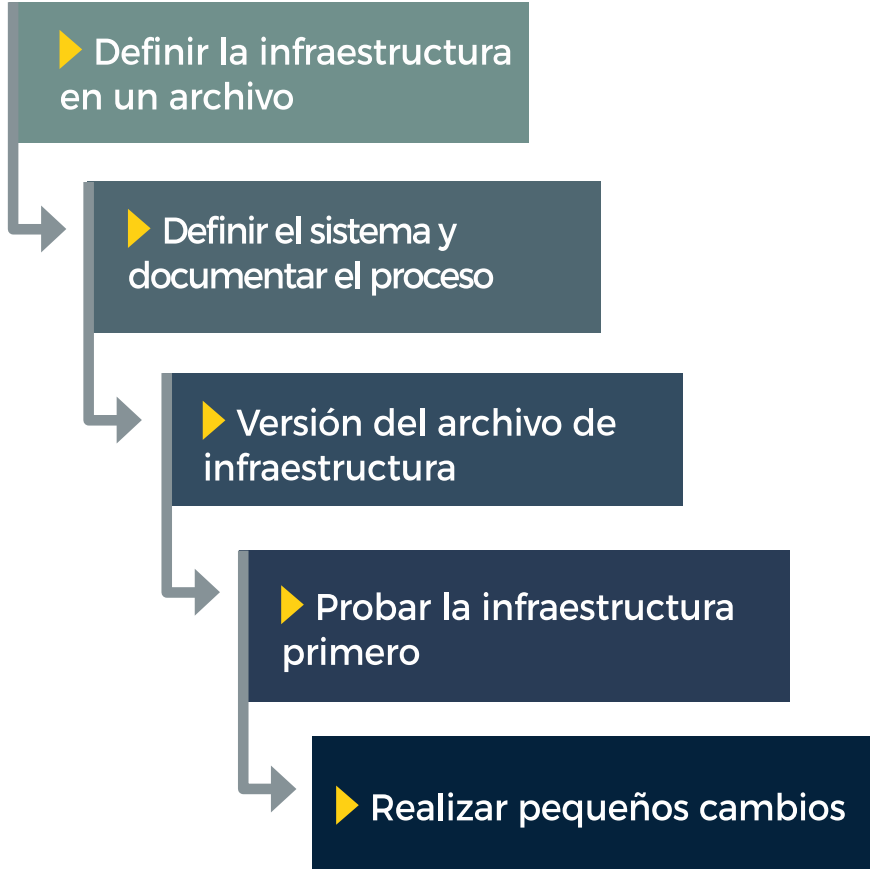
IaC. Infraestructura que se puede diseñar y lanzar mediante programación.



Para mover nuestra infraestructura hacia la IaC, debemos adherirnos a algunos **principios básicos:**

- Repetibilidad
- Consistencia
- Desechabilidad

Estos tres principios forman la **columna vertebral de IaC**, y hay que tenerlos en cuenta cuando se diseña la infraestructura. Además, hay que conocer los siguientes pasos:





Entidad Digital

Cuando se crea una entidad digital, se desea estar seguro de quiénes serán exactamente los usuarios. Para ello, se recopila un conjunto de entidades digitales directamente desde el navegador o celular.

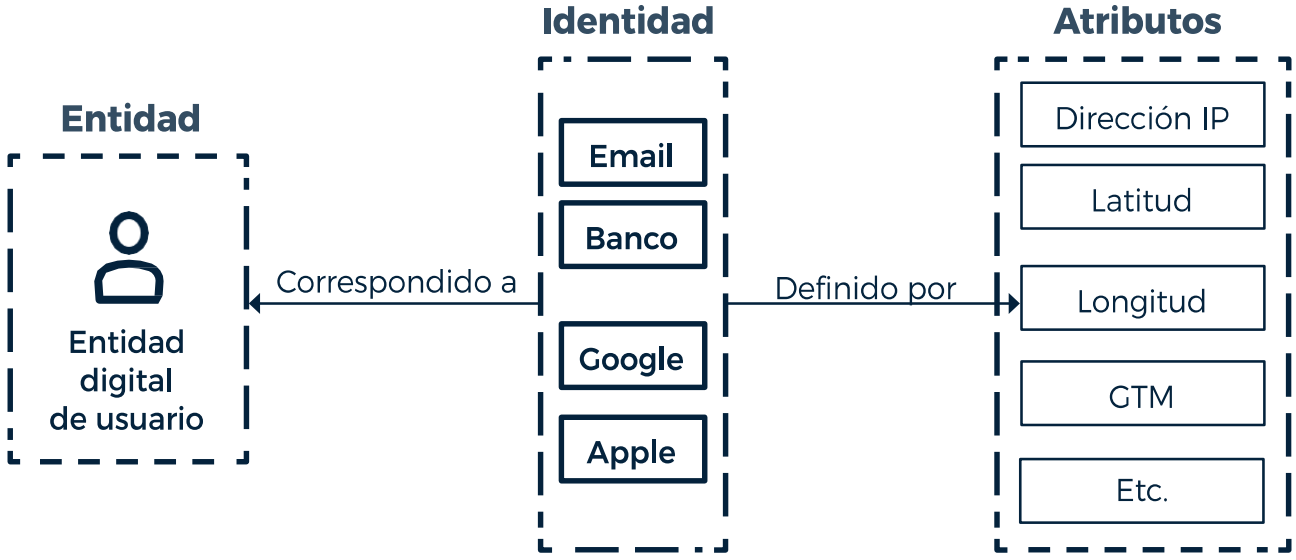
Se usan todos estos datos para identificar al usuario. Una vez recopilados todos los datos, cuando un usuario intenta una acción, por ejemplo, un nuevo inicio de sesión, el sistema compara los datos antiguos con los nuevos, utilizando patrones de convergencia específicos.

IAM y Entidad Digital

La IAM (Identity and Access Management) asume un papel muy importante en la definición y protección de la entidad digital. Un sistema IAM implica **cuatro funciones básicas:**

- Identificación pura
- Acceso de usuario
- Autorización
- Identificación federada

La identidad corresponde con una entidad digital para un usuario específico. Cuando se ha identificado al usuario, se puede usar la misma identidad digital para diferentes cuentas.



¿Cómo definir y crear nuestra **identidad digital**?

Un sistema IAM nace para gestionar al usuario. Esto significa que podemos crear, eliminar y actualizar su identidad. De esta manera, la entidad digital se convierte en un ADN digital, una cualidad única que nos permite movernos y navegar.

Los **principales componentes de un sistema IAM son**:



Autenticación

La autenticación es la fase en la que reconocemos a la entidad y verificamos qué o quién dice ser.



Autorización

La autorización es la fase durante la cual se define lo que puede hacer la entidad digital. Esto se delimita en el contexto de cada aplicación específica



Roles

Los roles son un conjunto de grupos y operaciones. Se utilizan para gestionar de forma centralizada las operaciones que realizan los usuarios.

Configuración y Administración de Redes en GCP

Cuando nos referimos a un entorno de nube, se habla de recursos compartidos a través de una red. Estos deben configurarse y administrarse, por lo que debemos garantizar el nivel de seguridad necesario para exponerlos y mantenerlos.

Para definir y administrar nuestra red, debemos identificar y comprender cómo está organizada. En GCP, la red se divide en dos componentes principales: **regiones** y **zonas**.





Introducción a Virtual Private Cloud

Una **VPC** brinda la flexibilidad de escalar y controlar la carga de trabajo del recurso a nivel global y regional. Cuando se conecta un recurso a GCP, se puede tener acceso a VPC sin tener que replicar la conectividad en cada región. Esto se debe a que la VPC lo hace por nosotros. No es necesario que una sola VPC de Google acceda a Internet para compartir el recurso entre diferentes regiones. Un único punto de conexión puede compartir el recurso entre regiones, lo que reduce la complejidad de la organización.

Firewalls y Routers

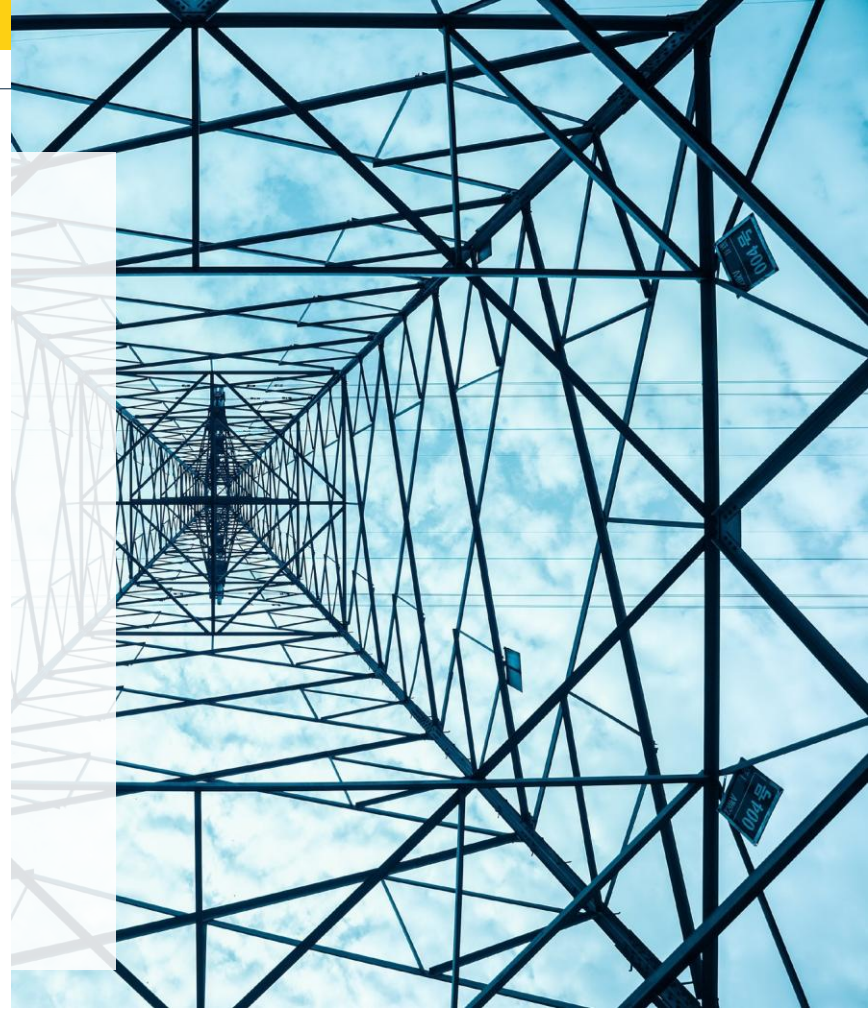
Cada red de VPC consta de uno o más rangos de IP. Estos se denominan **subredes**. Cada subred está asociada a una región.

Una red de VPC puede tener una o más subredes en una o más regiones. Esto significa que podemos hacer que nuestra red de VPC abarque diferentes ubicaciones geográficas para garantizar la alta disponibilidad de la nube. En este contexto, podemos definir la red de VPC **de dos formas**:

Modo automático	La red se crea automáticamente en diferentes regiones.
Modo personalizado	La red se crea sin ninguna subred asociada, lo que nos permite definir todas sus características.

Cuando se crea una red de modo personalizado, hay que diseñar los rangos de IP de acuerdo con **ciertas reglas**, las cuales son:

- Una subred debe tener un rango de direcciones principal, el cual debe ser un bloque CIDR RFC 1918 válido.
- Las subredes en la misma red deben tener un rango de IP único.
- Cuando definimos un bloque CIDR RFC 1918, debemos imponer algunas limitaciones en los rangos de IP que podemos crear.



Firewalls y Routers

Se definen como un par de direcciones que representan el destino y la puerta de enlace. Se usa una ruta cuando se quiere enviar un paquete a un destino específico, usando una puerta de enlace específica. Se puede identificar **tres tipos de destinos**:

- Anfitriones individuales
- Subredes
- Defecto





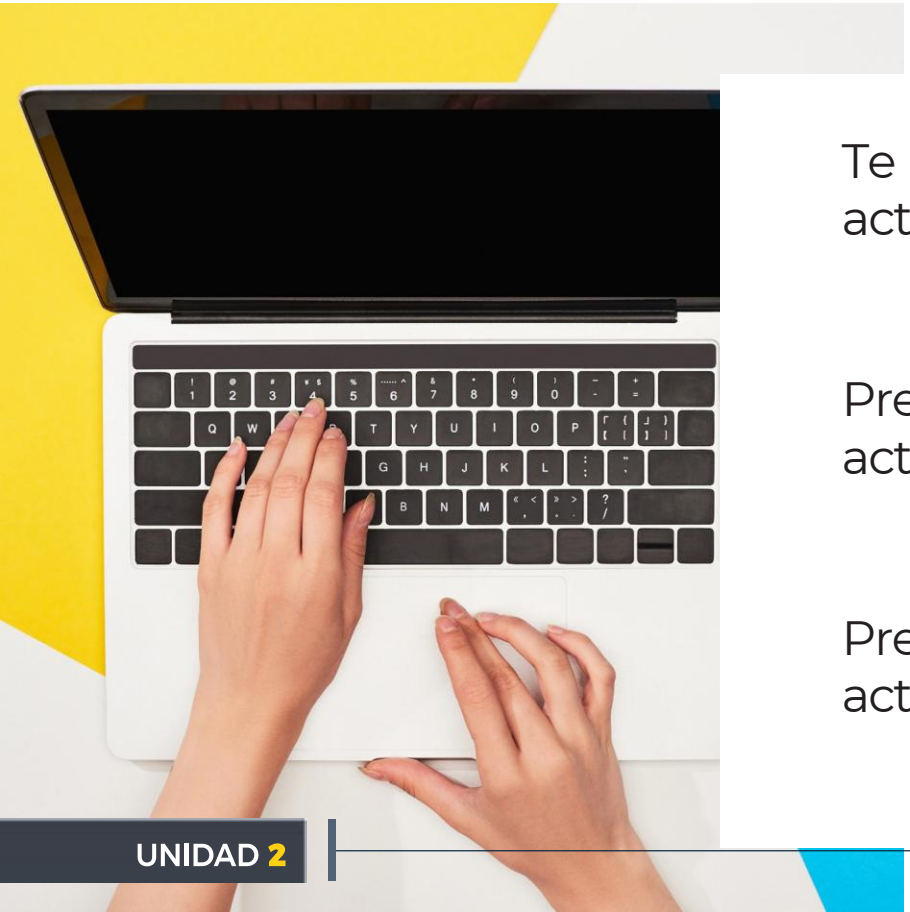
LECTURAS PARA REFORZAR LA UNIDAD

- Kodok, M. (2019). *Google Cloud Platform Solutions for DevOps Engineers.* Vilnius, Lithuania: @martonkodok.

- Murali Dodda, Vara Prasad Nulu. (n.d.). *A COMPLETE GUIDE TO GOOGLE CLOUD Platform: and why Google is winning the Cloud War.* Bitmin Infosystems.

- Vitthal Srinivasan Janani Ravi Judy Raj. (2018). *Google Cloud Platforms Architects.* BIRMINGHAM - MUMBAI: Packt Publishing.

ACTIVIDAD 1



Te invitamos a realizar la siguiente actividad:

Presiona el botón para descargar la actividad:

Presiona el botón para entregar la actividad:

CONCLUSIÓN



En esta unidad definimos las ventajas de utilizar DevOps con Google Cloud Platform. De la misma manera, conocimos con qué servicios podemos desarrollar nuestros proyectos, así como los beneficios del uso de la integración y entrega continuas.

Además, comprendimos más acerca de la arquitectura de los microservicios. En este contexto, un microservicio está diseñado para grandes sistemas, está orientado a objetivos y está diseñado para ser reemplazado. Adicionalmente, conocimos la implementación de una arquitectura de microservicios, así como su monitoreo, entre otros.

¡FELICIDADES!



Acabas de concluir la segunda unidad de tu curso *Servicios en la Nube*. Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente. Para ello, debes regresar a la pantalla principal y dar clic en *Presentar examen*.

3

UNIDAD

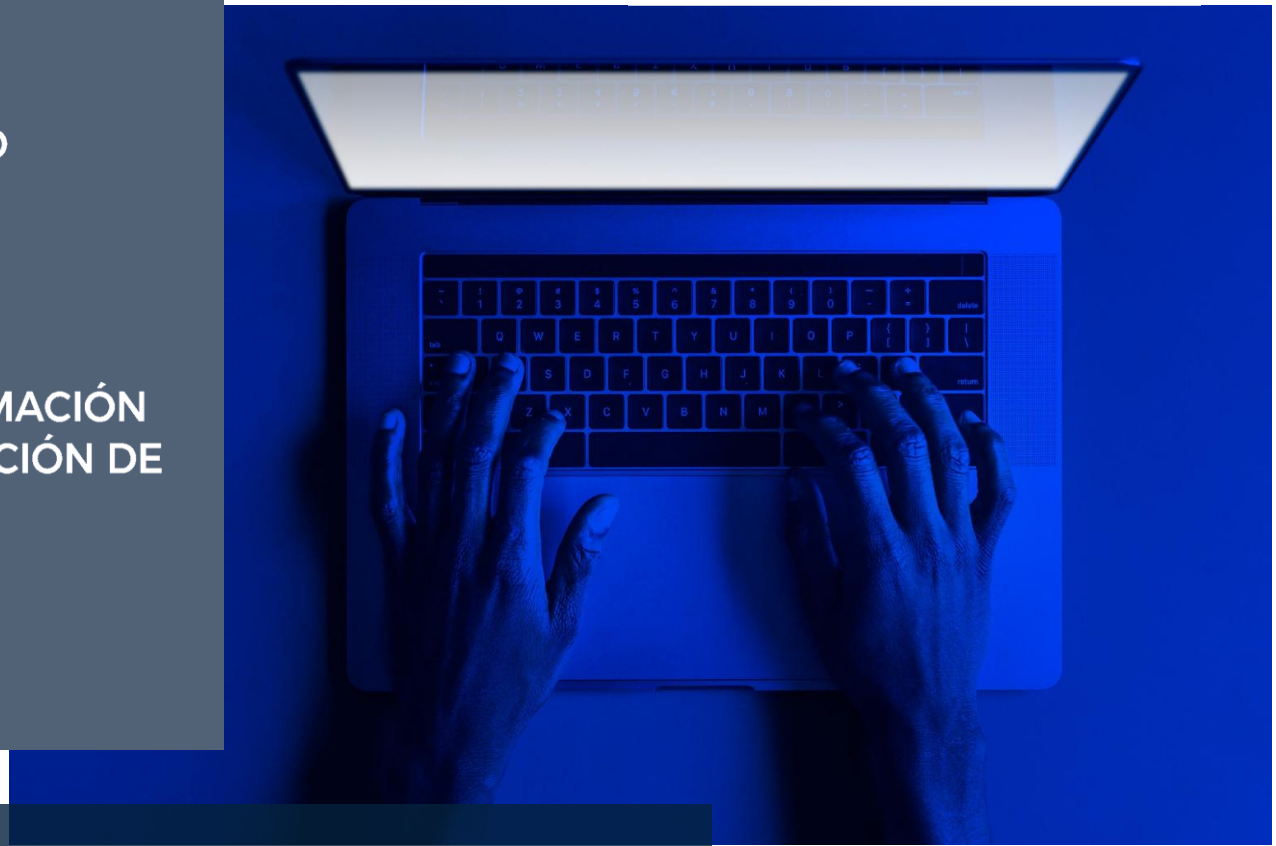


PLAYBOOK DE ADOPCIÓN DE DEVOPS I

TEMARIO

1.1 ADOPTANDO DEVOPS

1.2 TRANSFORMACIÓN Y OPTIMIZACIÓN DE DEVOPS





INTRODUCCIÓN

En esta tercera unidad aprenderás las jugadas que enseña el libro de aplicación de DevOps; es decir, el uso de un mapa de flujo de valor, la optimización de tiempo y recursos de un equipo de trabajo, entre otras.

COMPETENCIAS A DESARROLLAR



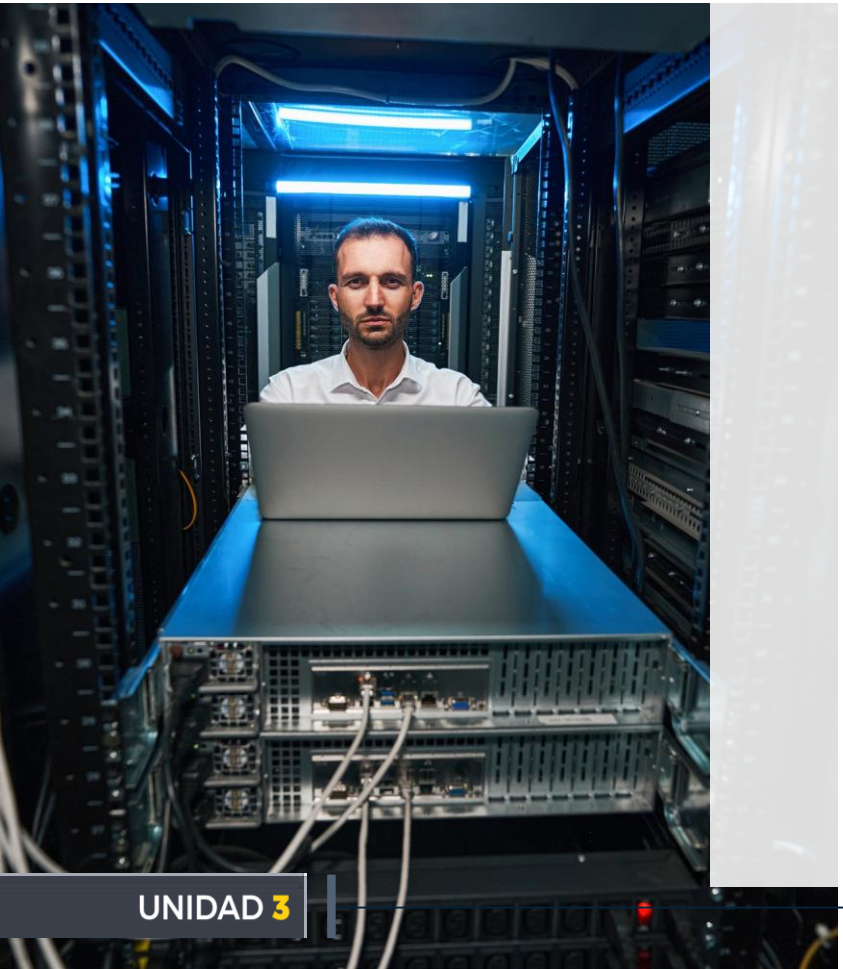
El alumno será capaz de comprender las jugadas que ofrece DevOps para una mejor optimización e innovación en el desarrollo de aplicaciones.



El alumno será capaz de entender que DevOps asegura que cada acción llevada a cabo en la entrega de software puede ser rastreada.

DevOps es un movimiento que nació en 2007 de forma accidental cuando Patrick Debois, un consultor *freelance*, estuvo analizando el sector IT. Su objetivo era ganar experiencia en todas las perspectivas en la cadena de valor IT. Debois estaba en un proyecto de consultoría para el gobierno de Bélgica para la migración de un *data center* y se frustró debido a los conflictos que se producían entre desarrolladores y administradores de sistemas.





Dev y Ops han convivido tradicionalmente en almacenes diferentes, con prioridades desalineadas e incluso opuestas:

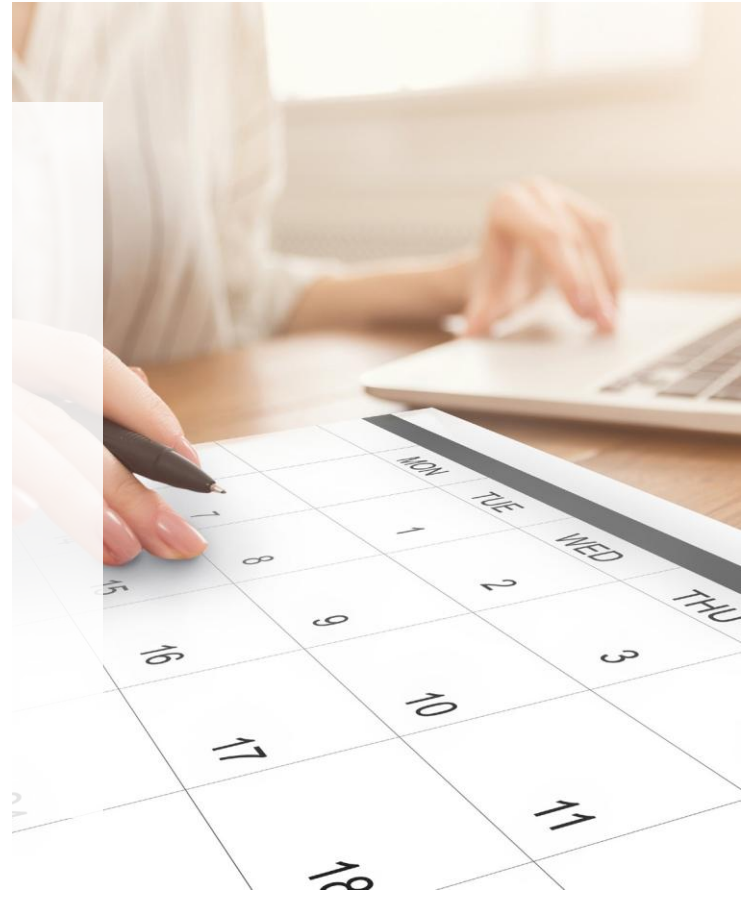
- **Dev** (Desarrollo). Tiene la tarea de crear innovación y ponerla en manos de los usuarios lo antes posible.

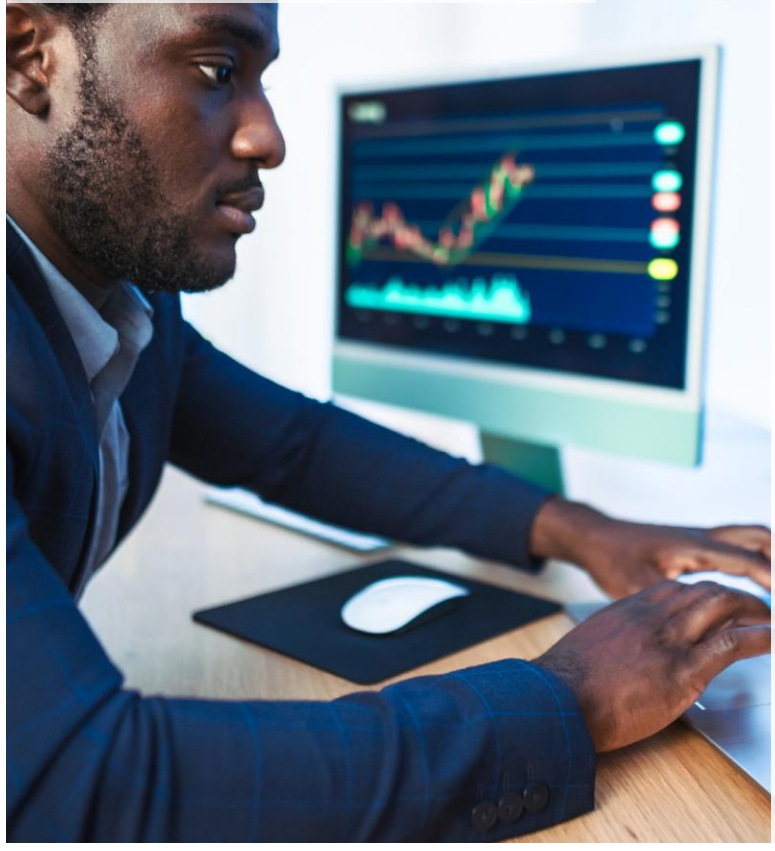
- **Ops** (Operaciones). Tiene la tarea de asegurarse que los usuarios tengan acceso a un sistema estable, rápido y receptivo.

Su objetivo en común es hacer del usuario un cliente satisfecho de los sistemas.

Antes de la llegada de *Agile*, en el paradigma orientado en cascada, las prioridades opuestas no eran un gran problema. Los desarrolladores y las operaciones trabajaban con un cronograma marcado por interacciones limitadas, solo en los momentos de lanzamiento.

Agile cambió todo eso con la Integración Continua (CI), ya que los desarrolladores ahora implementaban sus funciones a diario.



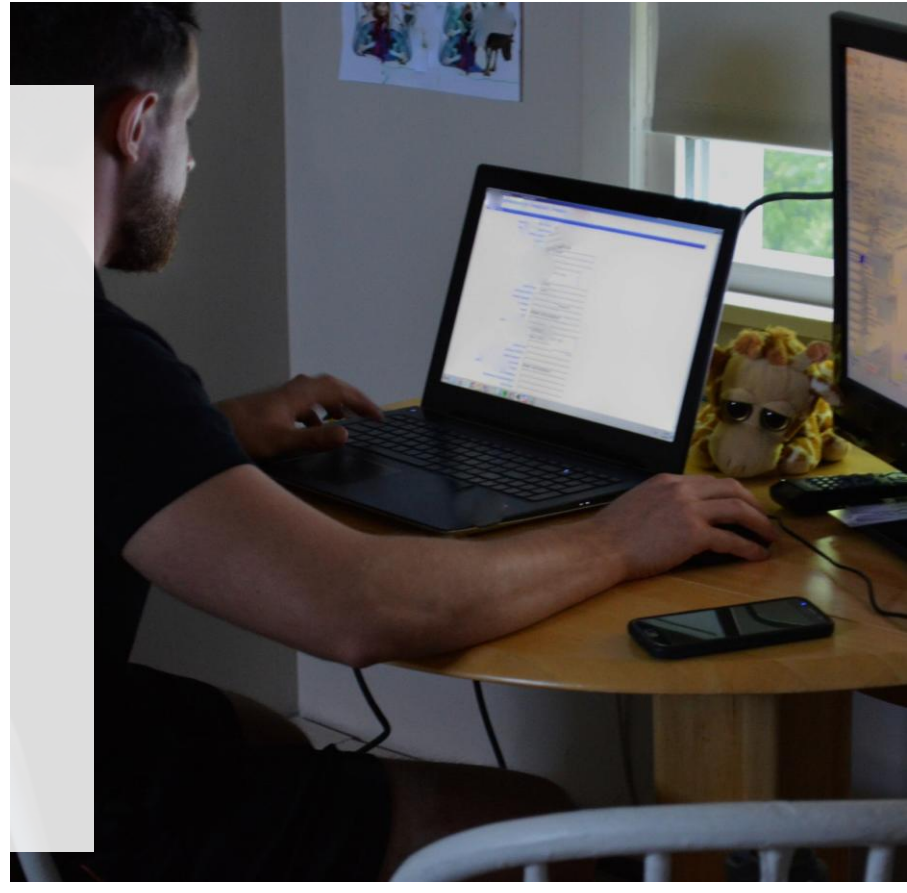


Punto de vista de los Dev:

- El desarrollador debe trabajar con Ops para comprender la naturaleza de los sistemas de producción en los que se ejecutarán sus aplicaciones.
- El desarrollador también necesita aprender a controlar las aplicaciones implementadas y comprender las métricas que le interesan a Ops.
- El desarrollador debe involucrarse más en las pruebas, tanto para probar la aplicación como para saber cómo funcionará en producción.
- El desarrollador necesita comunicarse y colaborar mejor con Ops.

Punto de vista de los Ops:

- Ops necesita saber qué código viene y cómo puede afectar a su sistema.
- Ops necesita automatizar la forma en que administran sus sistemas.
- Idealmente, Ops necesita versionar sus sistemas.
- Ops debe monitorear todo a lo largo del proceso de entrega, así como cualquier entorno que gestionen los equipos de operaciones.
- Ops necesita comunicarse y colaborar mejor con Dev.



DevOps: Prácticas

Varios líderes de opinión han dividido las prácticas DevOps en varias categorías y, en algunos casos, incluso con nombres diferentes.

IBM enumera varias de estas prácticas, que se encuentran en las siguientes **categorías generales:**

1. Piensa
2. Codifica
3. Entrega
4. Ejecuta
5. Gestiona
6. Aprende
7. Cultura

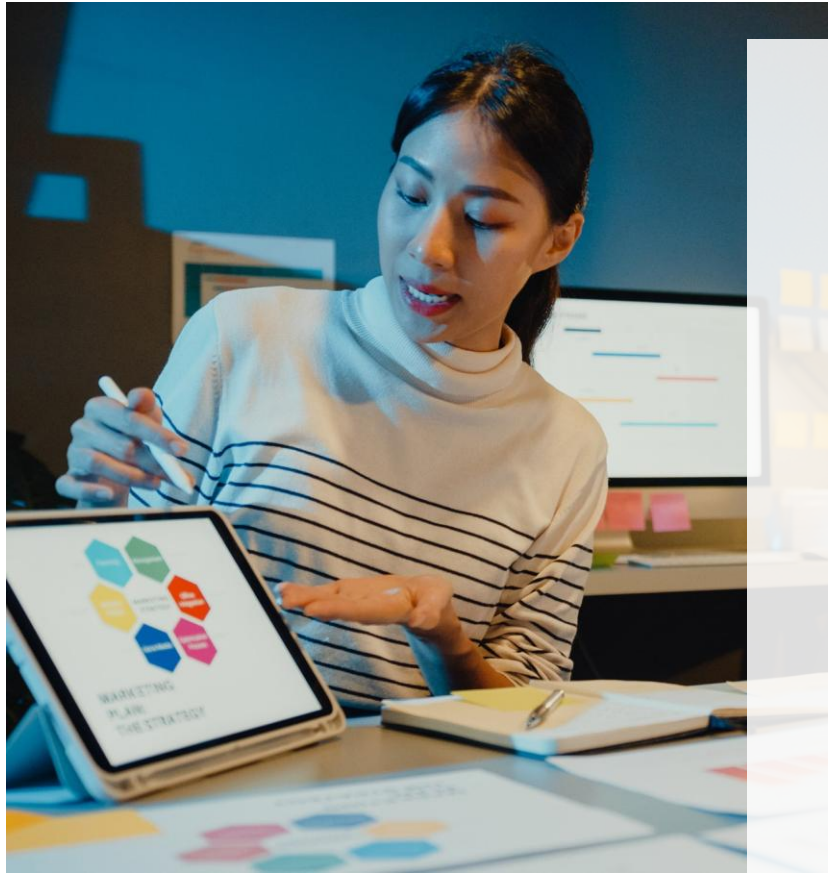
Para administrar y, en consecuencia, mejorar algo, es necesario utilizar **métricas** críticas. Una de ellas son los **Indicadores clave de rendimiento (KPI)**. Es necesaria una medición de referencia de estos KPI que marque el punto de partida, así como mediciones continuas para ver si realmente se está produciendo una mejora. Debe ser capaz de comprender la causa y el efecto: qué acciones dan como resultado la mejora de los KPI. Si están realizando varios cambios para las personas y los procesos, es fundamental saber qué cambios están resultando realmente en una mejora.



DevOps no se trata de adoptar un producto o un proceso. Se trata de **adoptar una filosofía**, que incluya principios y prácticas que incluyan a las personas, los procesos y las herramientas.

Adoptar DevOps no solo es adoptar un producto o proceso, es experimentar un cambio. La adopción de DevOps no es un proyecto de una sola vez, es un compromiso de **mejora continua** a través la aplicación de un conjunto de habilidades y prácticas basadas en los **principios Lean**:

- Eliminar el desperdicio
- Amplificar el aprendizaje
- Entregar lo más rápido posible
- Decidir lo más tarde posible
- Empoderar al equipo
- Embeber la integridad
- Considerar el sistema como un todo



Las **Líneas de Negocio (LOB)** necesitan que los departamentos de TI brinden capacidades a sus clientes y usuarios que les permitan obtener valor comercial. Los sistemas de TI no son más que el vehículo a través del cual las LOB entregan valor comercial a los clientes. Además, estas también dependen de dichos sistemas para recopilar comentarios sobre cómo los clientes consumen las capacidades comerciales y obtienen valor, entre otros.

Evaluación del Estado Actual

Comprender la madurez y el estado actuales de un equipo, proyecto u organización permite al liderazgo ejecutar de manera correcta la evaluación.

Para empezar a evaluar la madurez actual comencemos con algunas **preguntas fundamentales** sobre las capacidades de su organización:

1. ¿Puede entregar rápidamente aplicaciones nuevas e innovadoras y aprovechar las arquitecturas modernas?

2. ¿Puede modernizar las aplicaciones existentes para permitir la entrega e innovación?

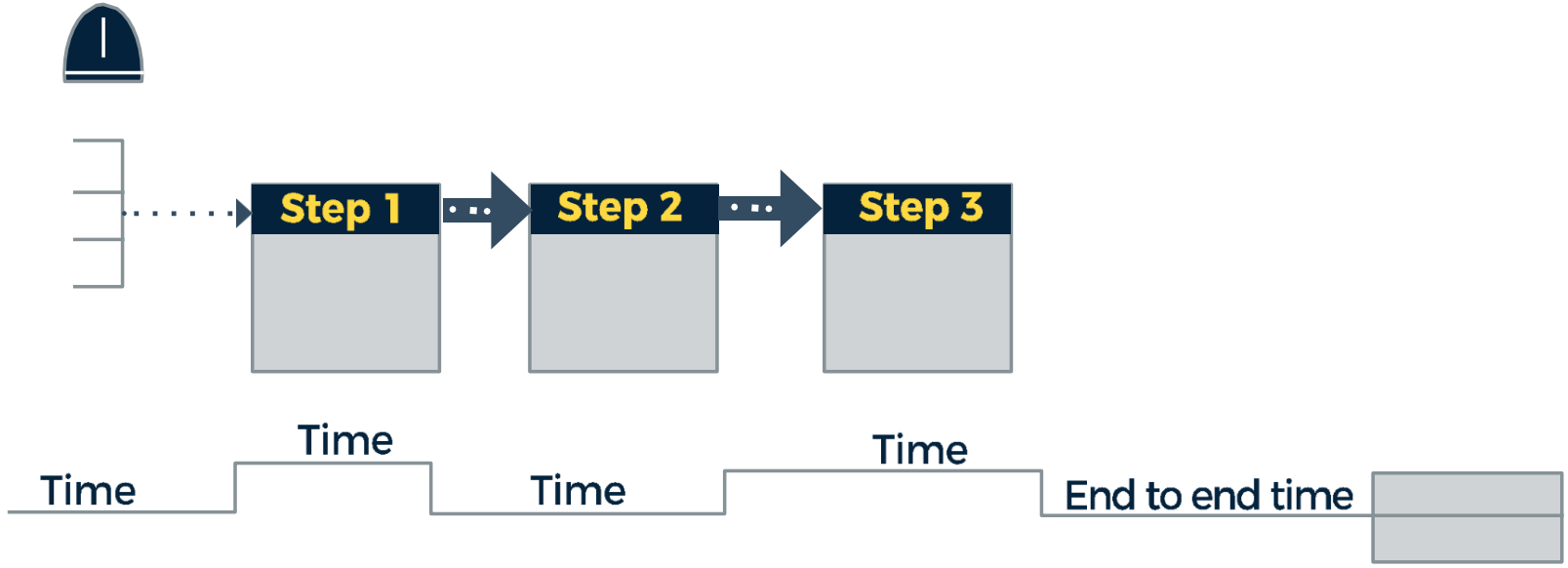
3. ¿Puede adaptar la cultura, las herramientas y los procesos para ayudarlo a tener éxito?

Mapeo del Flujo de Valor

Hay que identificar la ineficiencia o el desperdicio observando los canales de entrega de las diversas aplicaciones que ofrece una organización para buscar dónde hay desperdicio. Este último se presenta en varias formas. Entre los **ejemplos de desechos** se incluyen:

- Pasos del proceso innecesarios
- Retrabajo innecesario
- Funciones innecesarias
- Construir el artefacto o proceso incorrecto
- Transformar un artefacto construido por otra persona
- Esperar a que otra persona actúe en una acción o tarea
- Esperar una aprobación
- Esperar un entorno
- Crear un ticket para una tarea manual
- Crear o actualizar un artefacto que no agrega valor al usuario final o al cliente
- Informar a la gerencia mediante la actualización manual de bases de datos u hojas de cálculo
- Informe de estado

El método más eficaz para identificar el desperdicio en la tubería de entrega es el **Mapeo del Flujo de Valor (VSM)**. En el siguiente diagrama se muestra un mapa de flujo de valor simple y sus componentes:





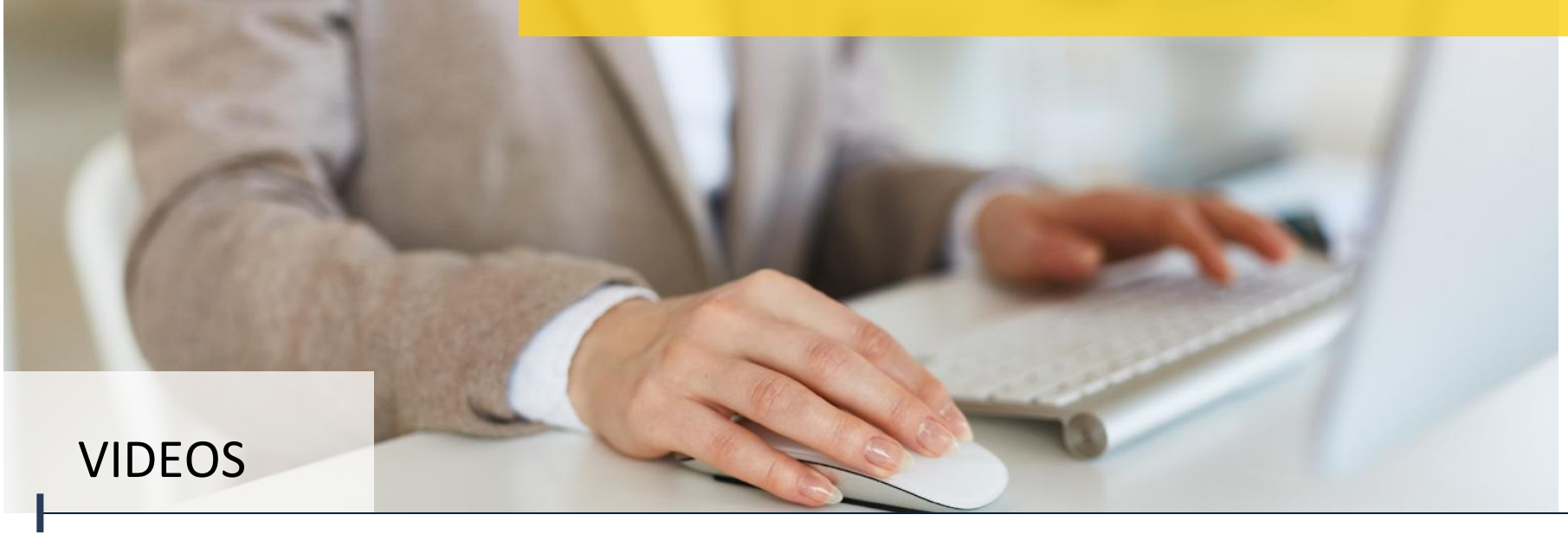
Una **cadena de suministro de software** extiende esta noción de cadena de suministro a la entrega de software y sistemas. Si bien la lógica empresarial subyacente y el valor de adoptar un modelo de cadena de suministro sigue siendo el mismo que para una cadena de suministro de fabricación, los paralelos no se aplican por completo. Para un fabricante de software, tiene mucho sentido subcontratar partes de su cadena de suministro.

Por otra parte, es importante identificar la **causa de los cuellos de botella** durante un ejercicio de mapeo de la cadena de valor.

Para ello, se necesita un **análisis de la causa raíz** para identificarlos. Un método típico para llevar a cabo un análisis de causa raíz es preguntar "por qué" cinco veces. Este enfoque permite superar los síntomas iniciales y llegar a la fuente o causa raíz.

A continuación, los **problemas más comunes de un cuello de botella**:

- La falta de integración de herramientas genera tiempos de espera a medida que los artefactos pasan de un profesional a otro.
- Los miembros del equipo carecen de visibilidad y sincronicidad con el trabajo de otros miembros.
- Es difícil aprender y contribuir más allá de los límites del equipo.



VIDEOS

Te invitamos a ver el siguiente video:



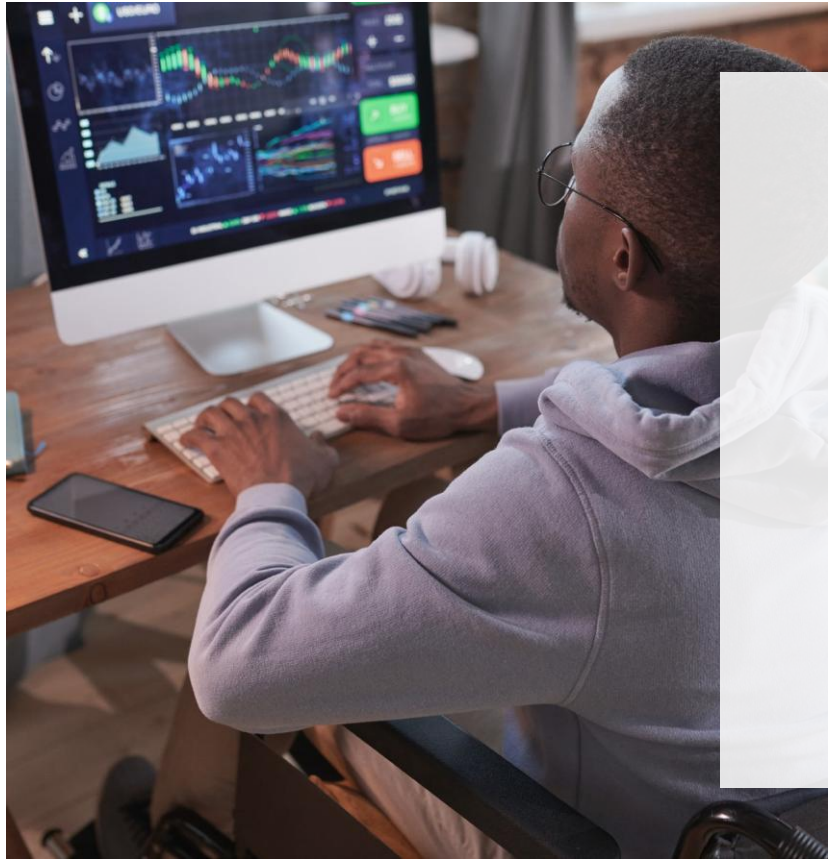
Business Case

Es un hecho que las capacidades de DevOps hacen que la canalización de entrega de aplicaciones sea ágil y eficiente al optimizar los procesos, las herramientas, las plataformas y la cultura. Sin embargo, las empresas requieren un caso de negocios con números tangibles para mostrar cuál será su Retorno de la Inversión (ROI). Existen varias **técnicas para llegar a tales cifras de valor comercial:**

Estudios de caso. Los proveedores de soluciones han realizado varios estudios de casos sobre cómo sus herramientas, plataformas y servicios de transformación DevOps brindan valor comercial y ROI.

Estudios de analistas. También se han realizado varios estudios por firmas de analistas externos. Forrester, por ejemplo, realiza estudios de Impacto Económico Total (TEI) para herramientas y prácticas.





- **Mapeo del flujo de valor.** Es una herramienta de Lean. Es útil para comprender, gestionar y mejorar el flujo de materiales e información dentro de una organización.

- **Evaluación del valor comercial.** La BVA es un enfoque formal para desarrollar un mapa de valor de las capacidades de DevOps que se están adoptando para asignarlas a retornos financieros tangibles.

Intención Comercial: Optimización vs. Innovación

Es importante comprender que la forma en que se ejecute depende no solo de los cuellos de botella que estás tratando de abordar, sino también de la intención comercial central de aplicaciones en las que se está realizando la obra.

Las organizaciones tienen **dos tipos de intenciones comerciales:**

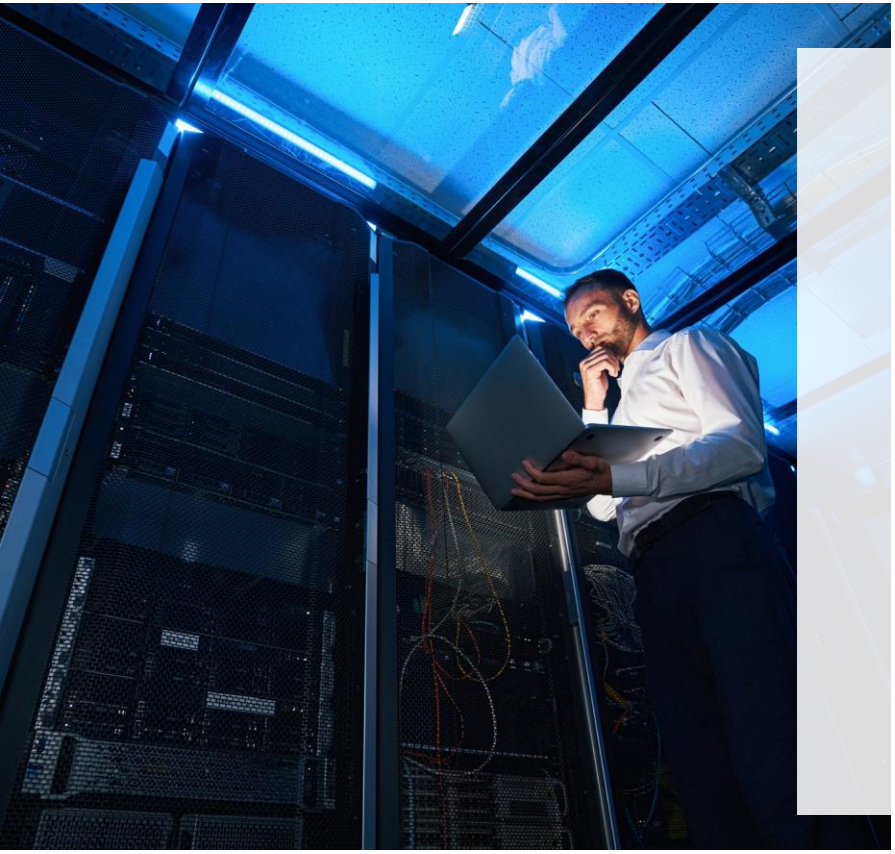




Según estos dos tipos de intención comercial, las **aplicaciones de una organización** se pueden dividir en **dos categorías**:

- El núcleo industrializado que ofrece capacidades comerciales centrales, es decir, lo que mantiene el negocio en marcha. Aquí, la principal intención comercial es la **optimización**.

- La ventaja de la innovación, donde se produce la experimentación con el fin de explorar e identificar nuevos modelos comerciales y métodos de participación del cliente; en otras palabras, la principal intención empresarial es la **innovación**.



Núcleo Industrializado

Las aplicaciones que caen dentro de la amplia categoría de núcleo industrializado son los **sistemas centrales de una organización**. Ellos son los que mantienen el negocio en marcha. Los modelos comerciales que se ofrecen aquí se comprenden bien, así como los métodos para interactuar con los usuarios. Estas aplicaciones suelen ser grandes y complejas, ya que la mayoría ha existido durante un tiempo.

Innovación

Las ventajas de la innovación se logran por medio de aplicaciones que se han desarrollado o se están desarrollando para impulsar la innovación:

- Experimentar con nuevos modelos comerciales
- Capturar nuevos mercados
- Experimentar con nuevos modelos de participación de los usuarios
- Aprovechar las nuevas tecnologías y arquitecturas modernas de entrega de aplicaciones.



Existe un conjunto de **temas centrales** que tienen sus raíces en *Lean* y forman la base de la mayoría de las obras de DevOps.

Antes que nada, es esencial comprender los siguientes **temas** y apreciar su importancia como núcleo de DevOps:



Minimizar el tiempo de ciclo. Para la entrega de software no existe una medida de progreso real. Por lo tanto, una entrega continua de software brinda la oportunidad de obtener retroalimentación de clientes reales o, en algunos casos, sustitutos del cliente que utilizan el software y brindan retroalimentación. Esta se puede utilizar para **mejorar tres cuestiones**:

- El software entregado
- El entorno en el que se entregó el software
- El proceso mediante el cual se entregó el software

Reducir el tamaño del lote. La entrega incremental es esencial para lograr procesos ágiles y eficientes. Para obtener el máximo rendimiento y lograr el menor tiempo de ciclo, el tamaño del lote debe administrarse correctamente. En este sentido, la adopción de Lean se enfoca en **tres áreas** (Reinertsen, 2009):

1. Minimizar el inventario.
2. Gestión explícita de los atrasos.
3. Hacer coincidir los tamaños de los lotes con la capacidad del equipo.

Establecer la cultura adecuada. Aquí, es importante la **Ley de Conway**, la cual dice que: "Las organizaciones dedicadas al diseño de sistemas [...] están abocadas a producir diseños que son copias de las estructuras de comunicación de dichas organizaciones". Esta afirmación se basa en el hecho de que dos módulos o subsistemas no pueden interactuar correctamente a menos que los diseñadores de los dos subsistemas se comuniquen entre sí.





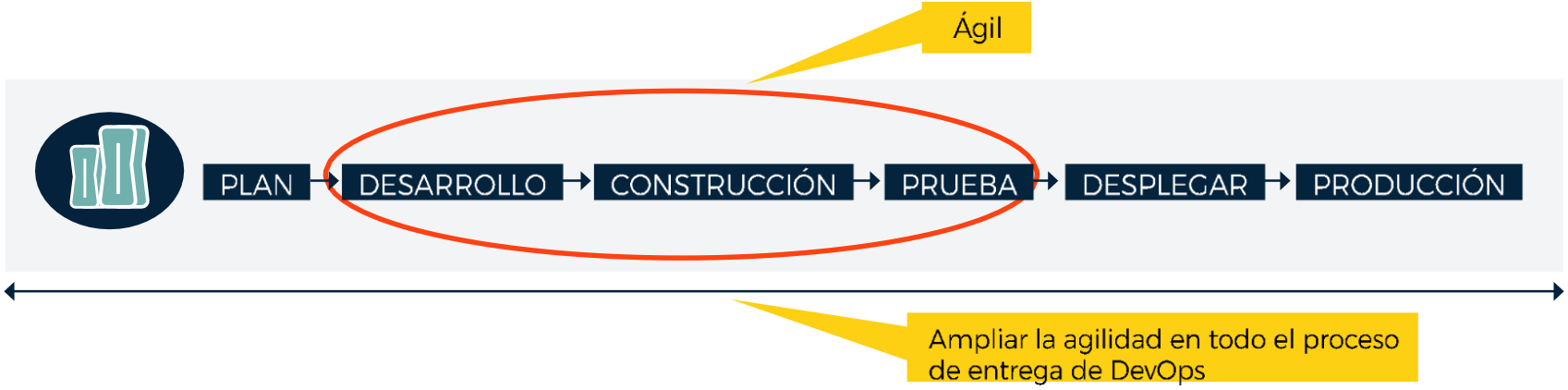
Jugada: Adopción Ágil

La mayoría de las organizaciones comienza la adopción de DevOps con proyectos que ya han adoptado *Agile*. Es, en esencia, un **requisito previo** para la adopción de DevOps. Algunos incluso han argumentado que DevOps extiende la adopción de *Agile left* hacia la gestión de proyectos: el inicio del proyecto, la fase de diseño y las operaciones.

DevOps comienza con *Agile* cuando los equipos que practican procesos de desarrollo en cascada pueden adoptarla.

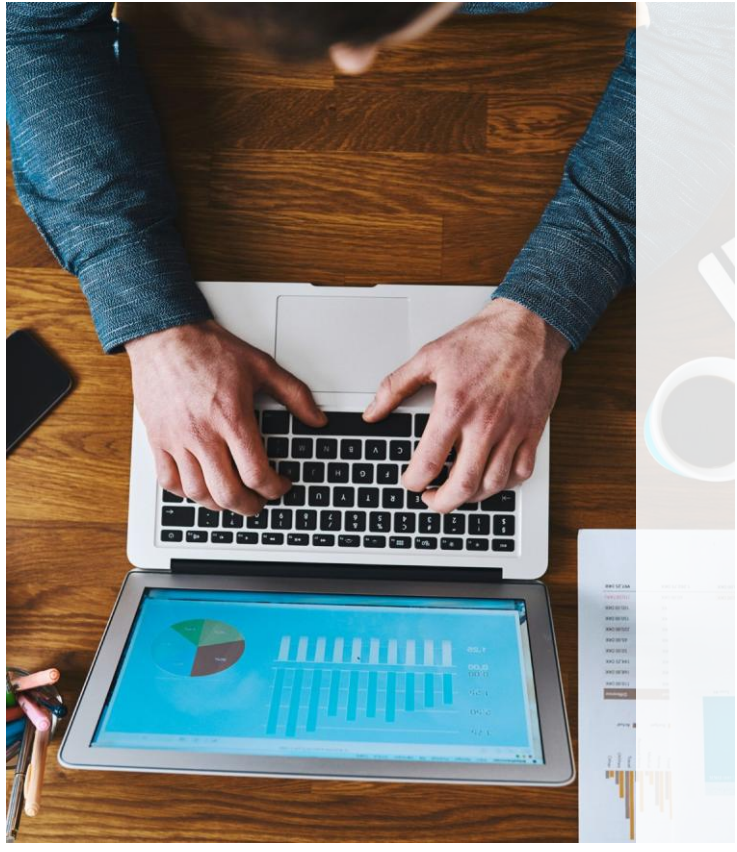
Desde la perspectiva de DevOps Play, hay **dos formas** en que se puede adoptar Agile:

- **Equipos ágiles.** Si tienes equipos o proyectos que ya han adoptado *Agile*, busca los que hayan madurado lo suficiente como para comenzar a alcanzar el límite de su productividad debido a las limitaciones que les impone el *Water-Scrum-Fall*.
- **Equipos no ágiles.** Estos equipos ciertamente pueden beneficiarse de la adopción de ciertas prácticas de DevOps.



Hay **dos tipos de entornos** en los que puedes implementar aplicaciones:

Estático:	Son entornos de estado estable. Tienen un número fijo de nodos, con conexiones de red predefinidas y estáticas entre ellos, formando una topología estática.
Dinámico:	La cantidad de nodos cambia con el tiempo, dependiendo de varios factores que gobiernan su estado. Los nodos se aprovisionan y desaprovisionan, según sea necesario. Si bien la red entre los nodos está predefinida, las topologías son de naturaleza más dinámica.



Los principales desafíos y fuentes de ineficiencia en el proceso de entrega tienen que ver conque los desarrolladores y evaluadores no tienen acceso a entornos que se parezcan ni remotamente a los entornos de producción reales. Esto puede hacer que las aplicaciones se desarrollen y prueben en entornos que funcionan y se comportan de manera muy diferente en entornos de producción, cuando finalmente se implementan allí. El proceso de implementación real puede estar lleno de errores si nunca se ha probado en un entorno similar al entorno de producción.

Full Stack

La implementación *Full Stack* significa, por consiguiente, implementar todas las capas que están en el diagrama de abajo:

En la implementación que **no es de *Full Stack***, todo lo que se implementa son las capas superiores. El resto de las capas inferiores se consideran parte del entorno y se aprovisionan antes de la implementación de la aplicación.





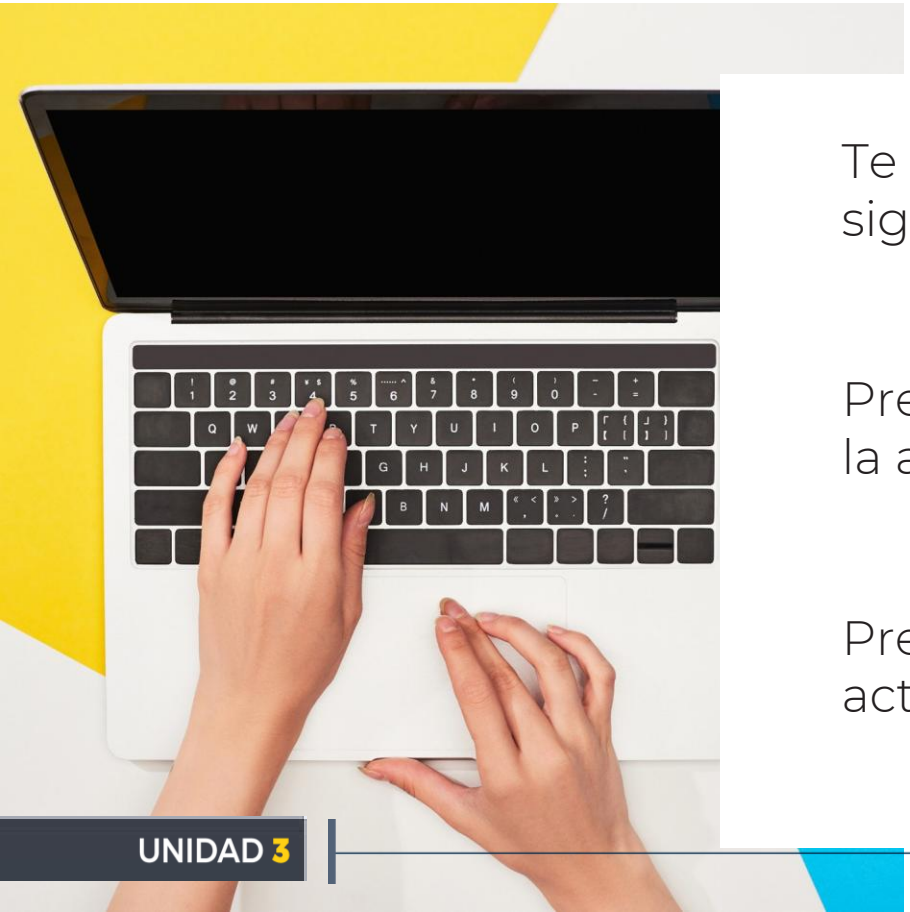
LECTURAS PARA REFORZAR LA UNIDAD

- Anand Srivatsav Amaradri, Swetha Bindu Nutalapati. (n.d.). *Continuous Integration, Deployment and Testing in DevOps Environment.* Karlskrona, Suecia: Blekinge Institute of Technology.

- Eric Onderdelinden, Mark Maijs, Donald Pondman, Balázs Nagy, Boris Smits, Marlies Quekel. (2020). *Deolitte. Retrieved from DevOps Point of View: An Enterprise Architecture perspective:*

- GARCIA, P. A. (2019). r*epository.eafit.edu.co. Retrieved from Prácticas Devops De Entrega Continua De Software Para La Transformación Digital De Los Negocios.*

ACTIVIDAD 2



Te invitamos a realizar la siguiente actividad:

Presiona el botón para descargar la actividad:

Presiona el botón para entregar la actividad:

CONCLUSIÓN



Con DevOps aprendimos que se debe adoptar el cambio para lograr un mayor valor comercial. Por otra parte, cualquier interrupción da como resultado caídas en la productividad, y eso también es cierto para la adopción de DevOps.

La adopción de DevOps debe comenzar con el primer paso de identificar su estado actual, y seguir hasta completar sus objetivos comerciales. Con esto, los líderes deben conocer las debilidades y fortalezas de su equipo de trabajo, entender las jugadas correctas y ponerlas en práctica durante el desarrollo de un proyecto.

¡FELICIDADES!



Acabas de concluir la tercera unidad de tu curso *Servicios en la Nube*. Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente. Para ello, debes regresar a la pantalla principal y dar clic en *Presentar examen*.



4

UNIDAD

PLAYBOOK DE ADOPCIÓN DE DEVOPS II

TEMARIO

4.1 JUGADAS PARA IMPULSAR LA INNOVACIÓN

4.2 ADOPCIÓN DE LIDERAZGO





INTRODUCCIÓN

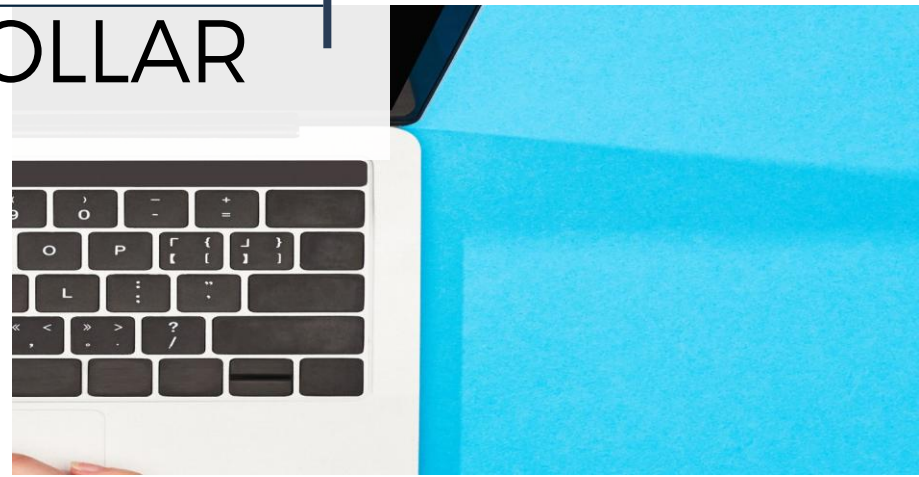
En esta cuarta unidad seguirás con el aprendizaje de las jugadas a ejecutar en un proyecto de desarrollo aplicando lo que nos ofrece DevOps.

Además, conocerás qué pasos debe seguir un líder de proyecto para tener un mejor desarrollo con base en la optimización e innovación.

COMPETENCIAS A DESARROLLAR



El alumno será capaz de comprender las jugadas que se deben ejecutar en el desarrollo de un proyecto.

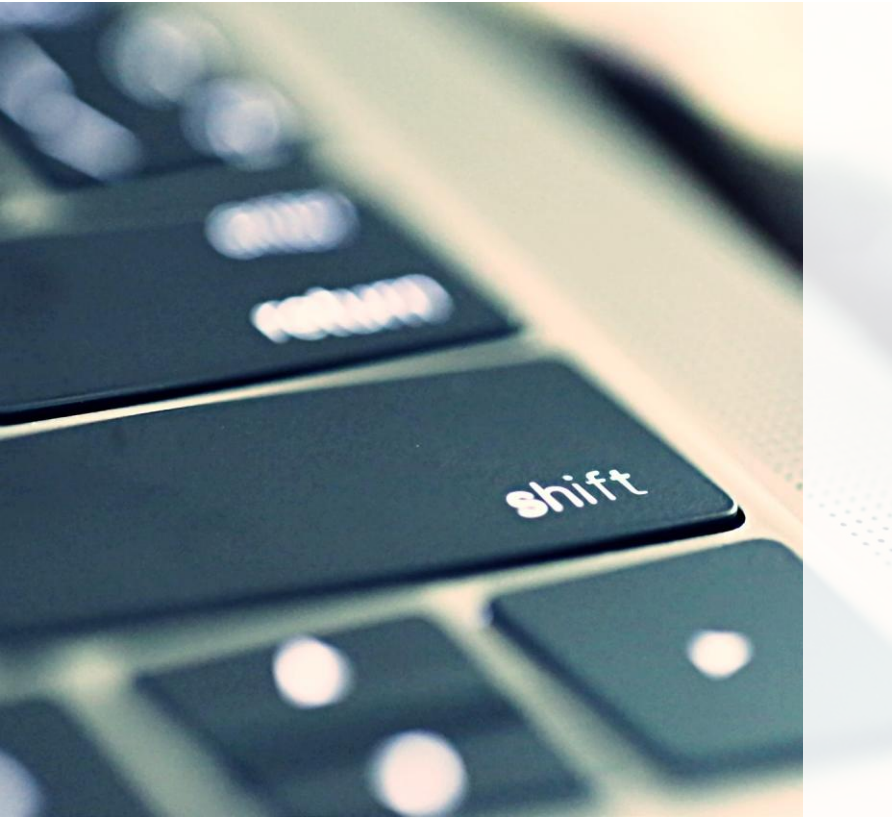


El alumno será capaz de conocer cómo liderar de manera correcta un proyecto con DevOps,

Jugada: Desplazar a la izquierda (Left testing)

Se refiere a mover las actividades de prueba al lado izquierdo del proceso de desarrollo y verlas como una línea de tiempo. Esto se debe principalmente a que la prueba, especialmente en el método de cascada, termina luciendo bien a la derecha. En la siguiente imagen podemos ver de dónde provienen generalmente las pruebas, después de que se realizan el análisis de requisitos y el diseño del sistema, y después de la codificación. Solo entonces aparece la fase del experimento.





Jugada: Desplazar a la izquierda (Shift Left)

Siempre ha existido una sana tensión entre los equipos de desarrollo y operaciones. De esta manera:

- **Dev** quiere poder tomar las aplicaciones y servicios nuevos o actualizados que han desarrollado e implementarlos en entornos estables y rápidos proporcionados por Ops, de manera continua.
- **Ops**, por otro lado, quiere estabilidad y control.

Debido a que uno de los objetivos de DevOps es la retroalimentación rápida de métricas y análisis a todas las partes interesadas en el proceso de entrega, es esencial establecer capacidades de **retroalimentación** y **monitoreo** continuos.

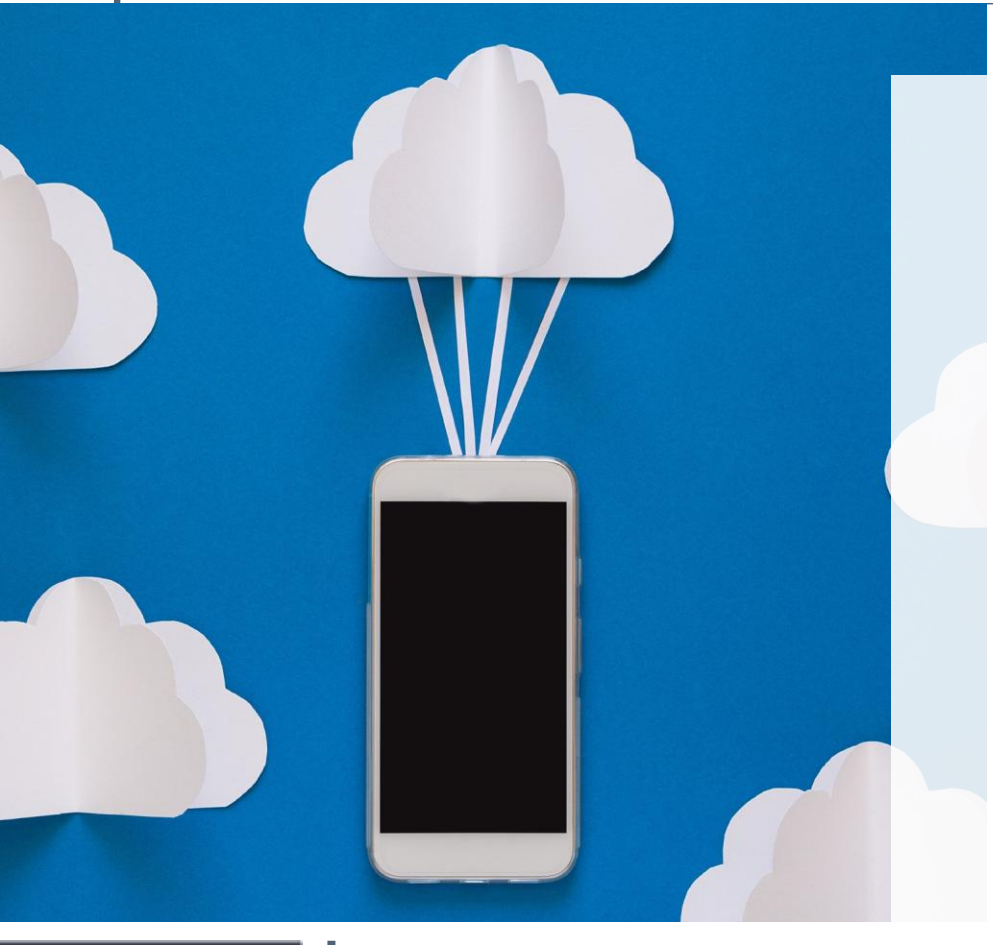
Gene Kim, gurú de DevOps, afirma que:

> “El objetivo de casi cualquier iniciativa de mejora de procesos es acortar y amplificar los ciclos de retroalimentación para que se puedan realizar las correcciones necesarias de forma continua”.

Con DevOps, el **lanzamiento a producción** debería ser tan simple como cambiar los ajustes de configuración para exponer una nueva versión de la aplicación que ya se ha entregado a un entorno de preproducción o incluso de producción.

Los procesos de entrega también deben ser ligeros, ya que se supone que se han dado pequeños lotes de la aplicación a través de la canalización de entrega a varios entornos.





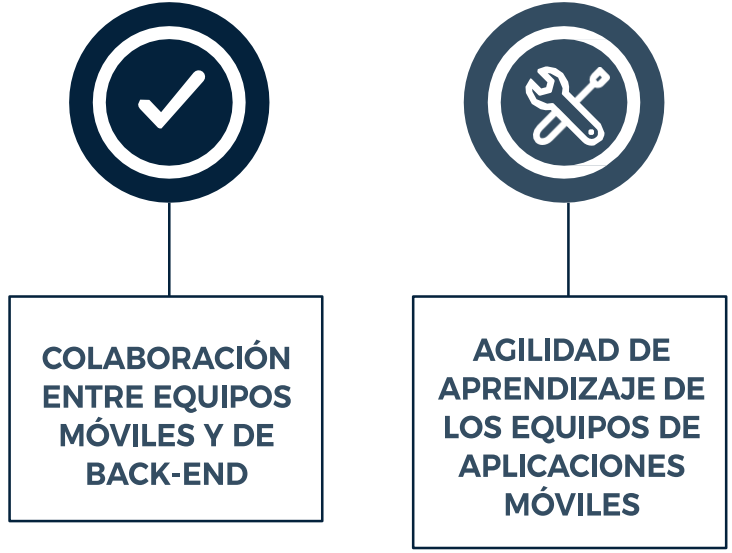
No existe realmente un DevOps separado para dispositivos móviles.

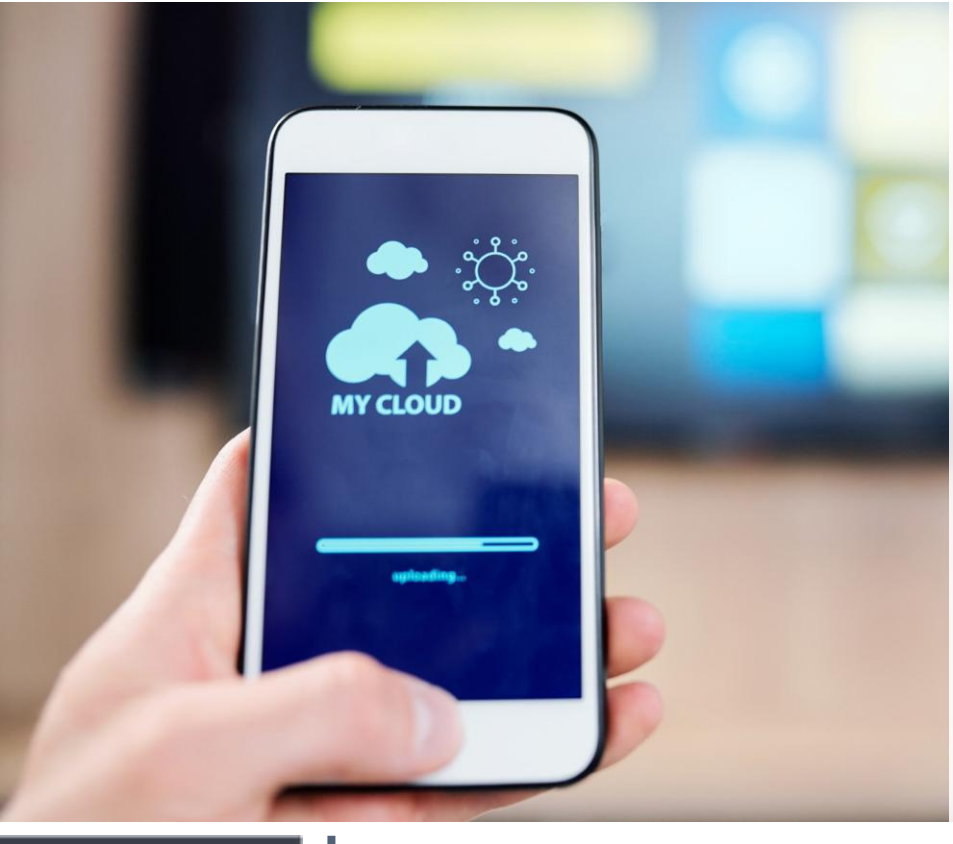
DevOps, como enfoque, funciona para todas las aplicaciones y componentes, desde las móviles de *front-end* hasta *middleware*, componentes de servidor de *back-end* y almacenes de datos. El objetivo es aplicar las prácticas y los principios de DevOps en ambos equipos (desarrollo y operaciones) para permitir la entrega continua de todos estos componentes, incluidas las aplicaciones móviles.

Comentarios sobre el sentimiento del usuario:

- **Entrega de apps móviles.** Las apps están a la vanguardia de la entrega continua. Es normal ver el dispositivo móvil y encontrar muchas apps que tienen nuevas actualizaciones.

- **Cultura y equipos.** En muchas de las organizaciones, los equipos de apps móviles han sido tradicionalmente pequeños, con sus propias herramientas y prácticas de desarrollo y entrega.

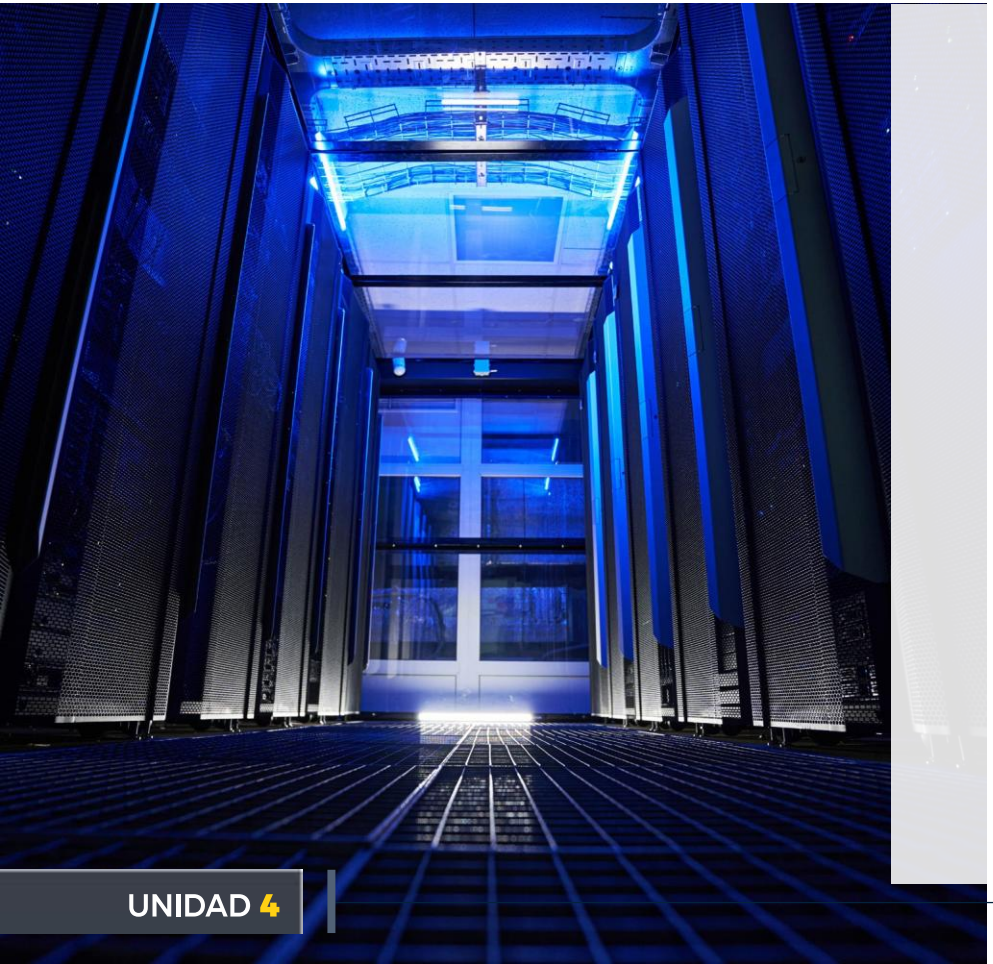




Jugada: DevOps para Internet de las cosas

Uno de los cambios más importantes en que la tecnología ha entrado en la vida cotidiana ha sido la llegada de los dispositivos inteligentes, conocidos como **Internet de las Cosas (IoT)**.

El IoT ha permitido que los dispositivos estén conectados, en tiempo real, con todos los demás dispositivos y servicios de *back-end* a través de Internet.



Jugada: DevOps para Big Data y Analytics

Las soluciones de *big data* y *analytics* representan una categoría completa de aplicaciones y servicios que están impulsando la demanda de enfoques nuevos, más rápidos y más frecuentes para la entrega de software. Esta tecnología se usa para analizar la gran cantidad de datos estructurados y no estructurados recopilados, organizados e interpretados por el software, transformándolos en información útil para la toma de decisiones y generación de *insights*.

DevOps para Impulsar la Innovación:

- **El síndrome Uber.** Se trata de un nuevo fenómeno que impulsa la necesidad de innovar en el mundo empresarial. Tiene que ver con actuar contra la amenaza percibida de un competidor que viene de la nada a su espacio, tal como lo hizo Uber. Esta necesidad de innovación se basa en un tema: ¡interrumpir o ser interrumpido!



Innovación y el papel de la tecnología. Un error común en el mundo actual es pensar que solo la tecnología es el impulso de la innovación. En este contexto, la tecnología rara vez se encuentra durante un proceso creativo. De esta manera, la función de la TI pasa a no ofrecer tecnologías disruptivas e innovadoras, sino una plataforma ágil, eficiente, predecible y estable en la que se puedan brindar los servicios empresariales disruptivos e innovadores.



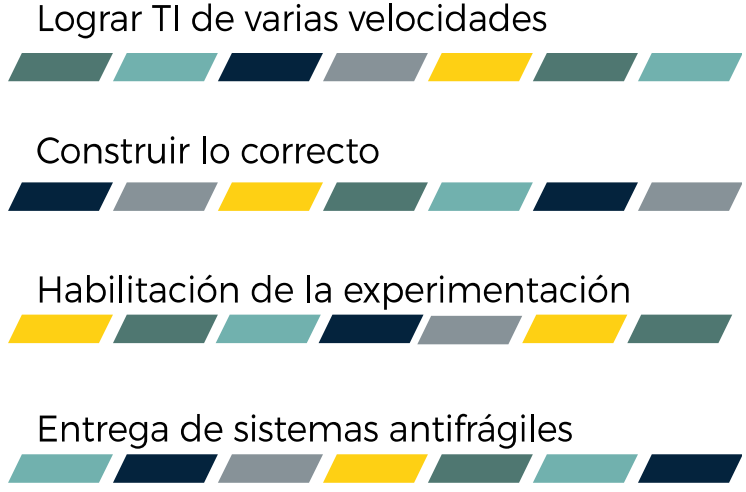


Innovación para nuevos modelos de participación del usuario. Se trata de otro camino común de innovación. En este sentido, los nuevos modelos pueden utilizarse para ofrecer nuevas capacidades comerciales a los clientes existentes o para capturar nuevos mercados. La experimentación vuelve a jugar un papel clave para descubrir nuevos modelos de interacción.

Jugada: Construir una Plataforma DevOps

Una plataforma DevOps ágil, resistente, escalable y confiable incluye tanto un punto de vista independiente de la tecnología como enfoques que son específicos de la misma. Combinemos esas ideas de construir una tubería de entrega integrada con la necesidad de sistemas antifrágiles.

En este sentido un enfoque de DevOps debe incluir:



Aplicaciones y Sistemas Antifrágiles

Uno de los requisitos principales de los entornos antifrágiles es la eliminación del tiempo de inactividad para las operaciones planificadas estándar, como el mantenimiento y la implementación de aplicaciones.

Los servicios ambientales y las aplicaciones que se ejecutan en los entornos tampoco deberían dejar de funcionar.

La construcción de entornos de alta disponibilidad requiere varias capacidades:

- Redundancia integrada en los servicios ambientales.
- Redundancia integrada en la arquitectura de la aplicación.
- Implementaciones azul-verde como parte del proceso de entrega continua.
- Monitoreo continuo de los entornos y aplicaciones para identificar problemas en tiempo real y mitigarlos si es posible.

Abstracción del Ambiente

El primer y principal objetivo de la creación de la plataforma DevOps adecuada (especialmente una que es antifágil) es abstraer las preocupaciones sobre el entorno y la infraestructura de los profesionales que desarrollan y prueban las aplicaciones y los servicios. Para ellos, el entorno debería aparecer como un conjunto abstracto de infraestructura o servicios de plataforma que pueden aprovechar y utilizar para desarrollar y entregar.





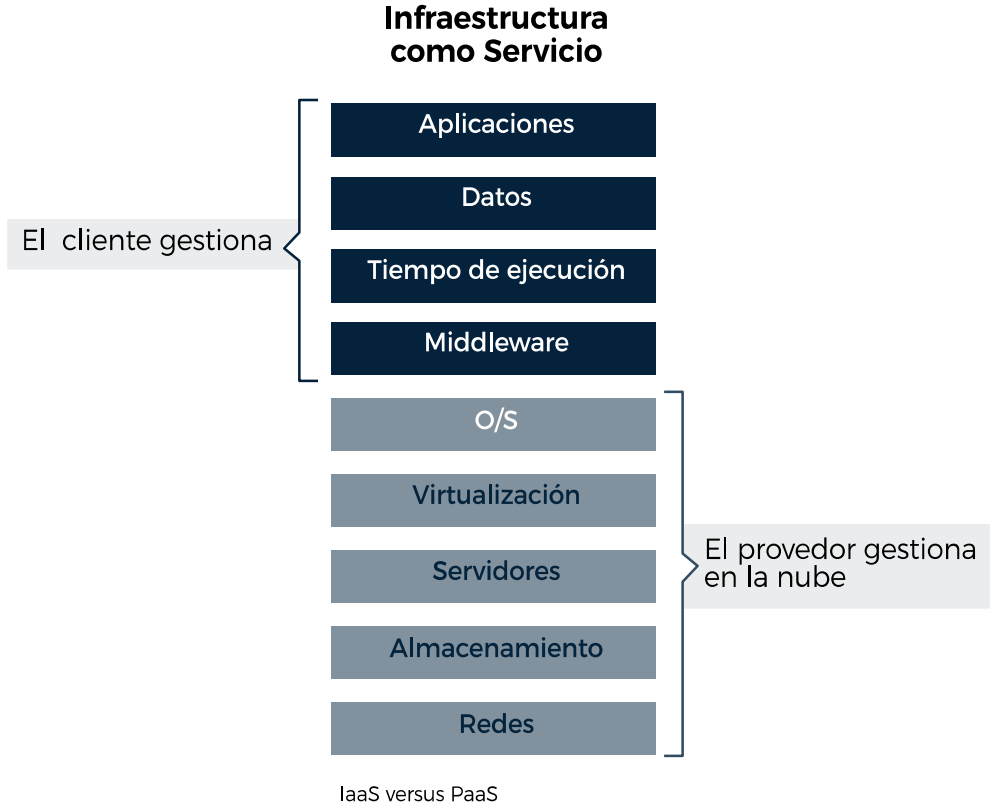
DevOps en la Nube

Un entorno alojado en la nube, ya sea entregado como IaaS, PaaS o contenedores ofrece un gran nivel de abstracción, lo que permite la entrega de una plataforma DevOps antifrágil.

A continuación, contrastaremos IaaS y PaaS. Además, veremos cómo entregar una plataforma DevOps en cualquiera de estos modelos.

Infraestructura como Servicio

En la imagen podemos visualizar la definición descrita por el Instituto Nacional de Estándares y Tecnología de EE. UU. (NIST). Muestra los componentes que están administrados por el proveedor de la nube en una oferta de IaaS, en comparación con los que no lo están. El proveedor de la nube aquí puede ofrecer los servicios en la nube o el propio equipo de operaciones de la organización, dependiendo de si la nube es administrada por el proveedor o autogestionada.

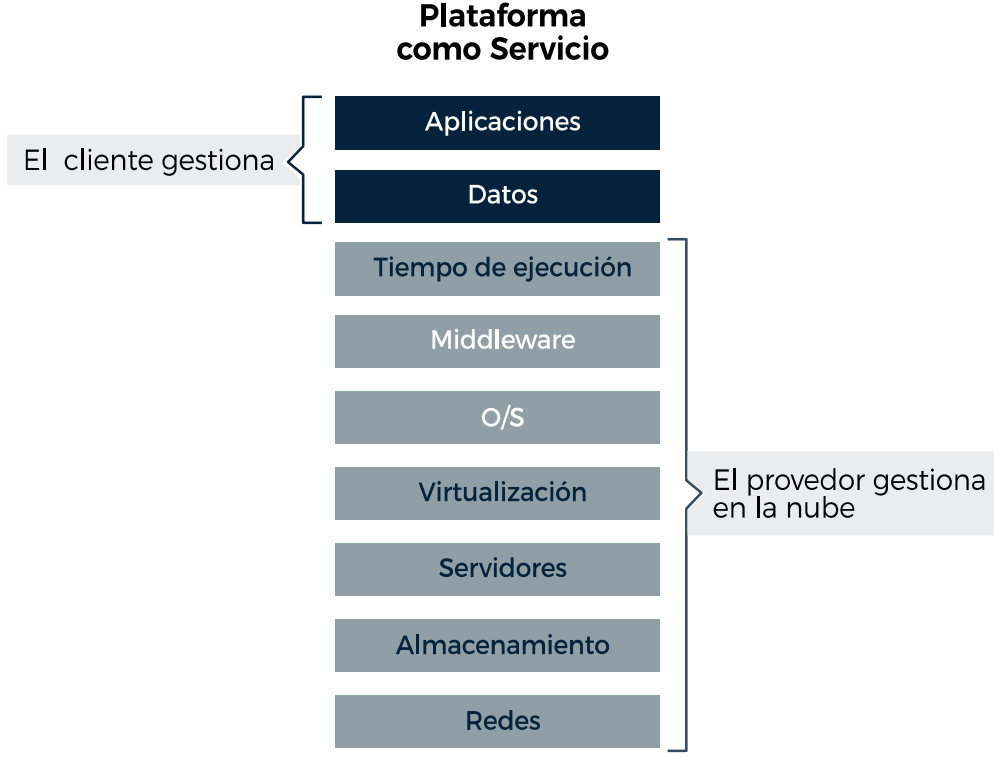


IaaS versus PaaS

Plataforma como Servicio

En la imagen podemos ver qué parte de la pila administra el usuario (o cliente) frente a cuánto administra el proveedor de la plataforma en la nube, que nuevamente puede ser un proveedor o el propio equipo de operaciones de una organización.

El objetivo de aprovechar una plataforma en la nube IaaS para estos entornos es agregar la capa de abstracción entre los profesionales que entregan software y la infraestructura.

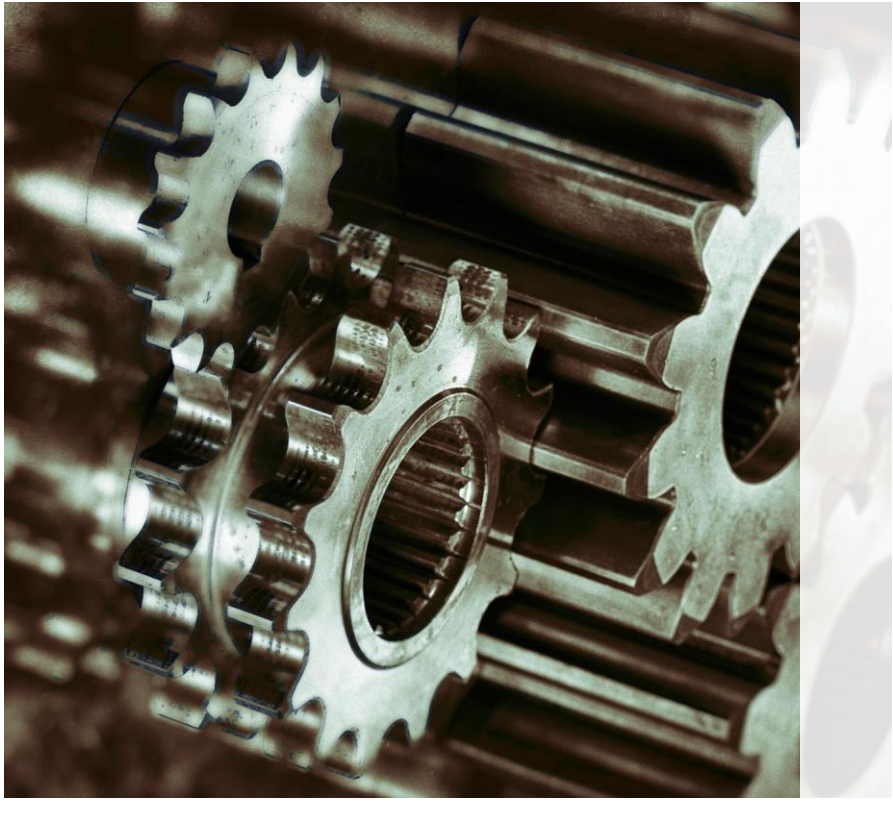


Servicios de DevOps con Paas

Hay que considerar los servicios que deben alojarse en la plataforma para implementar una canalización de entrega de aplicaciones DevOps. En esta, cada uno de los componentes que la conforman debe estar disponible como un servicio para proporcionar una solución DevOps en PaaS completa.

La siguiente lista nos muestra las **capacidades que componen una canalización**:





Jugada: Arquitecturas de Microservicios

Desde una perspectiva de DevOps, **la entrega de lotes pequeños es un tema central**, cuyo valor es evidente: pequeños cambios entregados con frecuencia, así como la reducción del tiempo del ciclo de retroalimentación y del impacto del cambio. Además, las pruebas y la validación de seguridad se vuelven más fáciles. En consecuencia, la implementación se vuelve más fácil a medida que se implementan conjuntos de cambios más pequeños.

Jugada: Desarrollar una Economía API

Las API representan el mecanismo predeterminado mediante el cual las aplicaciones y los servicios se comunican entre sí. Si se ha desarrollado una arquitectura basada en microservicios, este es el modo predeterminado. Las API se han convertido en un mecanismo para impulsar la innovación, al permitir a los desarrolladores crear nuevas aplicaciones, aprovechando las API internas y externas para agregar más servicios a sus aplicaciones. **Lo hacen de dos formas**:

Aprovechamiento del ecosistema. Las organizaciones pueden hacer esto mediante el desarrollo de aplicaciones nuevas e innovadoras que aprovechan las API de servicios externos de terceros.

Escalar a través del ecosistema. Las organizaciones ahora pueden monetizar sus propias capacidades comerciales más allá de las capacidades de monetización de sus propios modelos comerciales, al exponerlas a un ecosistema más amplio a través de API.

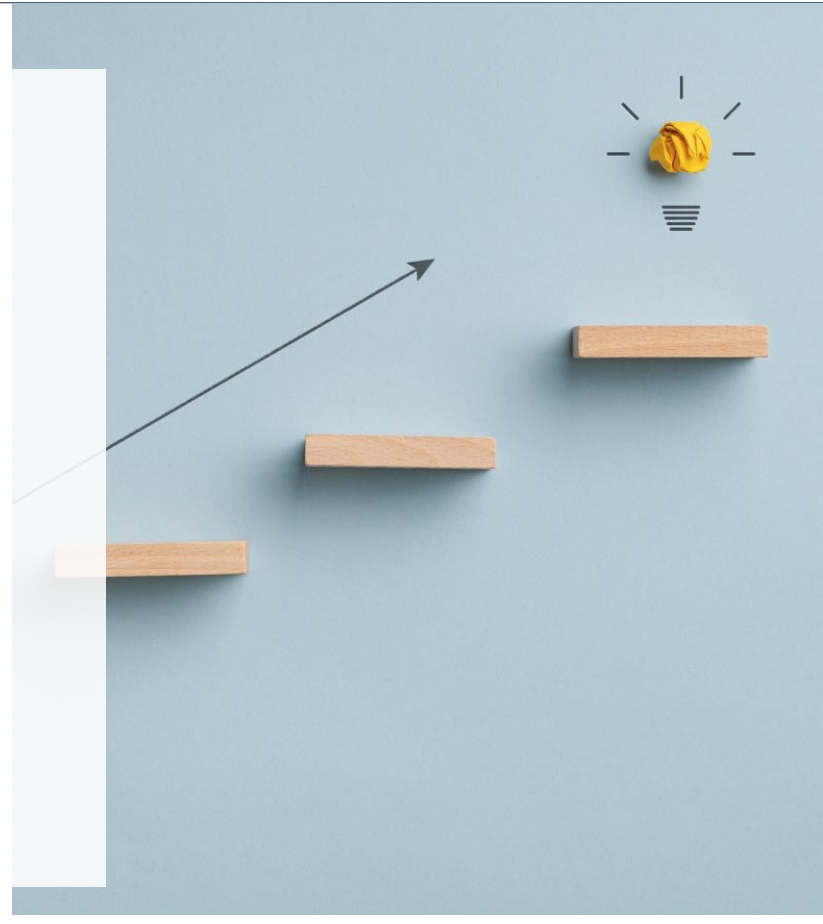


Jugada: Centro de Competencia de DevOps (CoC)

Un COC no se trata una organización administrativa o un grupo de herramientas/habilitación, sino un lugar donde los adoptados de DevOps vienen para aprender de los expertos, y unos de otros, con el fin de compartir la experiencia y las lecciones aprendidas. A medida que las organizaciones adoptan y escalan las prácticas de DevOps, el CoC también se convierte en una fuente de experiencia, habilitación, orientación de herramientas e incluso alojamiento de DevOps.

Jugada: Desarrollar una Cultura de Innovación a Escala

Como afirma el CIO de IBM, Jeff Smith, “el objetivo final es escalar una cultura de innovación en toda la organización”. Si bien DevOps permite esto, cambiar la cultura requiere una **transformación concertada en toda la organización**. Esta debe afectar la forma en que todas y cada una de las líneas de negocio, división, programa, proyecto, equipo e individuo abordan sus tareas diarias, así como la manera de abordar la entrega del valor solicitado por el negocio.



Jugada: Modelos de Equipo para DevOps

Para desarrollar un modelo de equipo escalable para DevOps se necesita **un equipo de partes interesadas multifuncionales** con los siguientes **parámetros**:

- Todas las habilidades multifuncionales.
- Pueden comunicarse y colaborar sin los silos organizacionales.
- Tienen visibilidad del trabajo de los demás.
- Pueden pasar artefactos de un lado a otro a través de una cadena de herramientas integrada.
- Son lo suficientemente pequeños como para fomentar la agilidad y la independencia necesarias para operar.
- Son escalables a equipos grandes y distribuidos al replicar el equipo pequeño en grandes cantidades.
- Los conjuntos de equipos pueden comunicarse y colaborar entre sí.
- Existen modelos de gobernanza generales para administrar los equipos a escala.



Jugada: Estandarización de Herramientas y Procesos

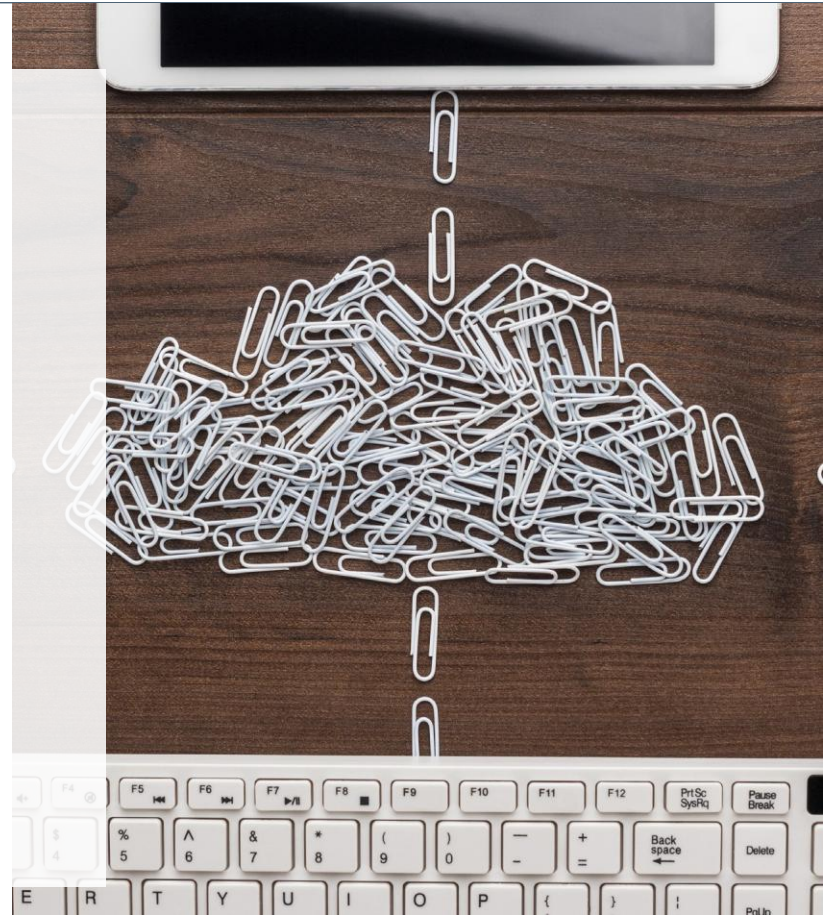
Hay ciertas herramientas que se comercializan por la forma en que son utilizadas por los profesionales. Estas tienen un impacto mínimo en otros profesionales y, por lo tanto, no necesitan ser estandarizadas o integradas.

Estandarizar los repositorios de código es esencial, ya que se convierten en la herramienta de colaboración para que los desarrolladores compartan código entre sus pares.

Jugada: Consideraciones de Seguridad para DevOps

- **Equipos de entrega de aplicaciones.** Están motivados para brindar capacidades nuevas e innovadoras rápidamente.

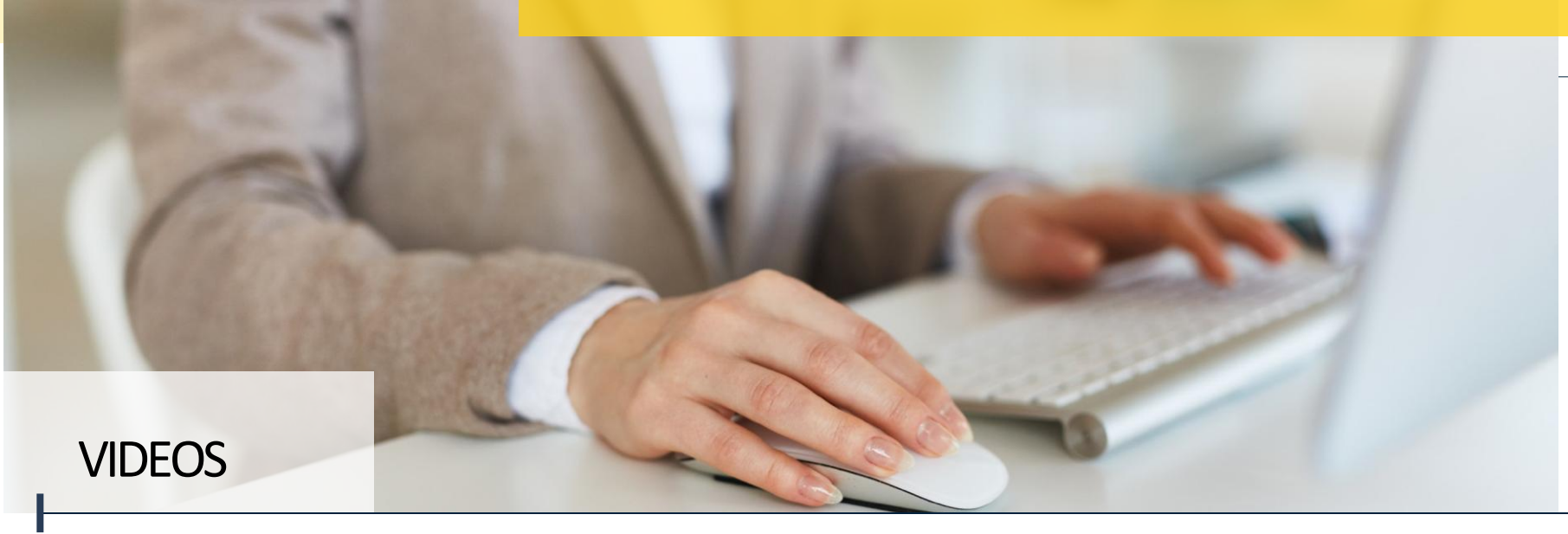
- **Equipos de seguridad.** Están decididos a garantizar que los nuevos sistemas y capacidades sean seguros, y que las funciones de seguridad sean sólidas.





Jugada: DevOps y la Subcontratación

Muchas organizaciones tienen al menos algunas de sus operaciones de TI o de entrega de aplicaciones subcontratadas a un proveedor externo. La subcontratación puede ser tradicional, donde el trabajo se descarga a un proveedor externo en el extranjero; o un verdadero modelo de cadena de suministro, donde los proveedores entregan componentes de la cadena de suministro de entrega de *apps*.



VIDEOS

Te invitamos a ver el siguiente video:

ADOPCIÓN DE LIDERAZGO

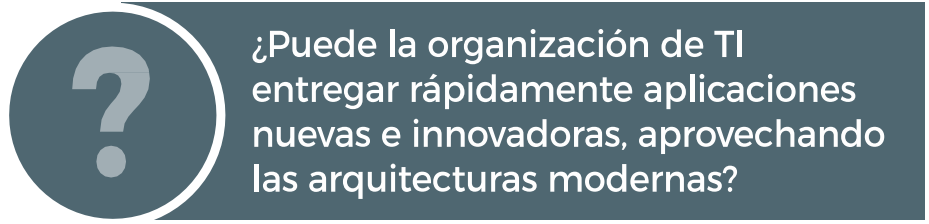


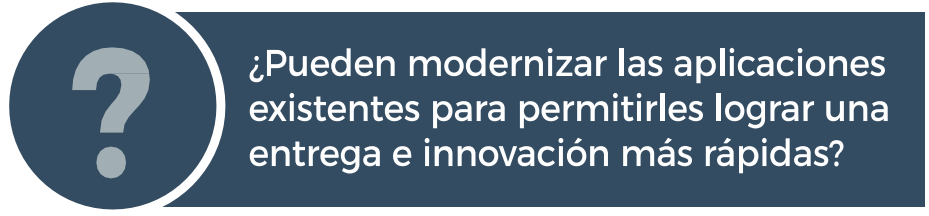
El papel del ejecutivo es **liderar desde el frente**. Algunas de su **características** son:

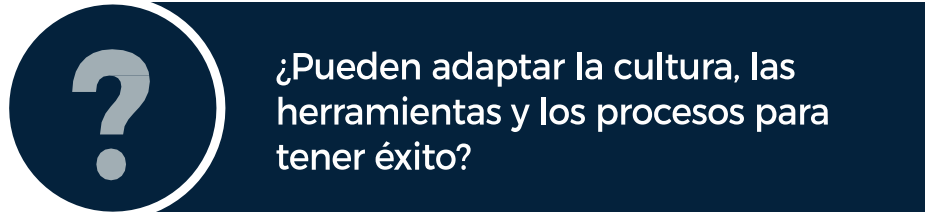
- Impulsar el cambio cultural.
- Tener la capacidad de dar un paso atrás e identificar las áreas de inercia cultural que se han establecido en la organización.
- Tener una visión de cómo debe verse la cultura objetivo.
- Capacitar a sus líderes para que realicen los cambios necesarios en los procesos, la gobernanza, las métricas y los objetivos comerciales a los que se dirigen.

Los líderes necesitan convertirse en partes interesadas de pleno derecho en el esfuerzo. Necesitan hacer preguntas para ayudar a la organización de TI a identificar la brecha entre sus necesidades comerciales y lo que pueden ofrecer, para luego trabajar con TI para ayudarlos a transformarse y llenar las brechas que identificaron.

Bajo esta perspectiva, existen tres **preguntas clave** que los líderes tienen que hacerse:

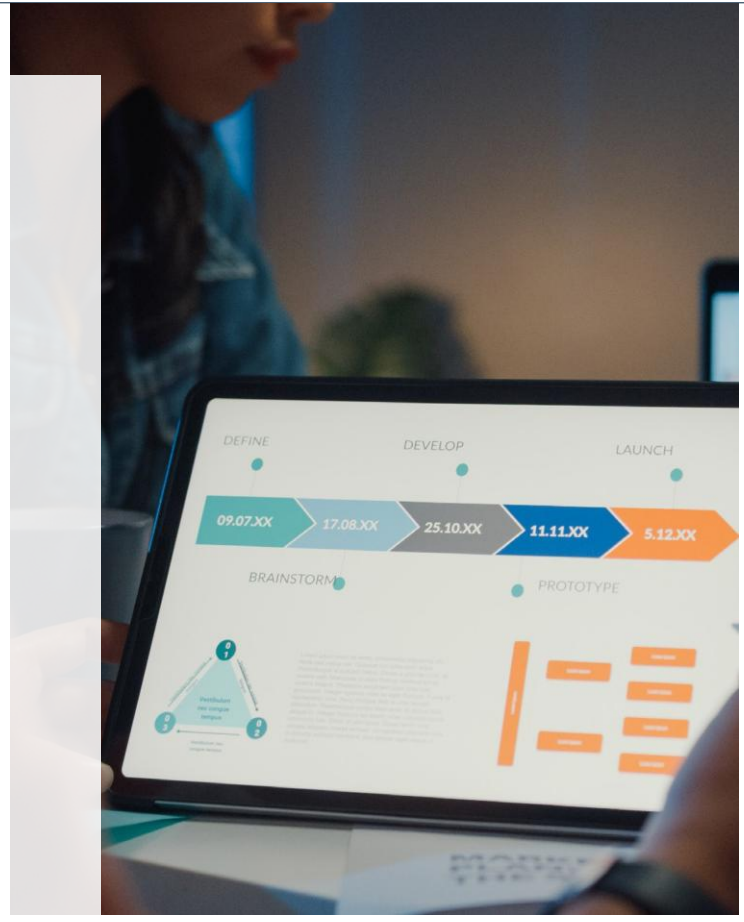






A medida que se desarrolle una hoja de ruta de adopción de DevOps, se definirá un conjunto de capacidades que deben adoptarse para abordar las áreas de desperdicio identificadas.

Los líderes deben adoptar estas capacidades individualmente o pueden utilizar las jugadas anteriores para adoptar conjuntos de capacidades juntos. Estas afectarán todas las áreas de DevOps: proceso, herramientas, plataforma y cultura.



Adicionalmente, un líder debe adoptar estas capacidades ejecutando experimentos y aprovechando un **proyecto piloto**.

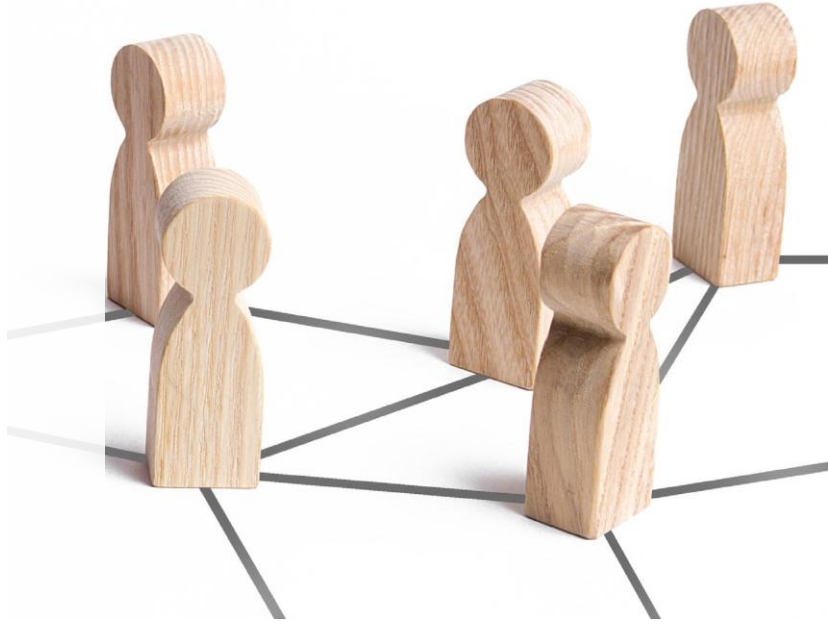
El enfoque ideal es identificar un proyecto piloto para cada capacidad que se adopte.

De esta manera, los **objetivos** de cada proyecto piloto son:



Ya sea que sean llamados inconformistas o *unicornios*, siempre hay personas en cada organización que se destacan. Los unicornios son personas innovadoras, ya que realmente *piensan fuera de la caja*:

- Son líderes inherentes.
- Pueden ser sofocados muy fácilmente por estructuras corporativas, políticas, jerarquías y gobierno.
- En lugar de convertirse en los colaboradores y líderes de equipo más productivos, pierden su encanto o se van y se llevan sus habilidades a otra parte.



Piensa en el **unicornio** como el **inconformista** y el **portaviones** como la organización. La única forma de fomentar unicornios es en una granja de unicornios y permitirles correr libres con otros unicornios.

Construir equipos que sean autoorganizados, capaces de operar libremente y que se diviertan, es de lo que se trata la construcción de granjas de unicornios. Una cultura de confianza y colaboración fomenta estas granjas, donde la comunicación libre supera a las jerarquías.

> “Si un unicornio camina hacia el Este en un portaviones, pero el portaviones se dirige hacia el Oeste, ¿dónde terminará el unicornio?”.



LECTURAS PARA REFORZAR LA UNIDAD

- DevOps, D. (n.d.). *DASA Devops Agile Skills Association.* Recuperado de: DASA DevOps Fundamentals: https://www.cognitus.com.mx/Content/temarios/DASA_DevOps.pdf

- Justin Vaughan Brown, Peter Matthews. (2015). *CA Technologies. Retrieved from Perspectivas de DevOps 2: Ideas, opiniones y orientación de líderes de DevOps.*

- Castellanos, Camilo; Correal, Dario. (2018). *Executing Architectural Models for Big Data Analytic.*

CONCLUSIÓN



Esta unidad se enfocó en las jugadas que representan patrones de éxito que se observan al adoptar DevOps para proyectos y programas que tienen como objetivo la innovación. En este sentido, la clave para tener éxito y poder escalar la adopción de DevOps más allá del equipo independiente a proyectos y programas a escala empresarial está en la cultura y el trabajo en equipo.

Para finalizar, el liderazgo se centra en lo que debe hacer la cabeza del proyecto para liderar una transformación DevOps en toda su organización: cultura de colaboración y confianza, comenzar con un proyecto piloto, criar unicornios en un portaviones, etcétera.

¡FELICIDADES!



Acabas de concluir la cuarta unidad de tu curso *Servicios en la Nube*. Te invitamos a finalizar este esfuerzo realizando el examen parcial correspondiente. Para ello, debes regresar a la pantalla principal y dar clic en *Presentar examen*.

BIBLIOGRAFÍA



- Riti, P. (2018). *Pro DevOps with Google Cloud Platform: with Docker , Jenkins and Kurbenetes*. Mullingar, Westmeath, Ireland: APress.

BIBLIOGRAFÍA



- Sharma, S. (2017). *The DevOps Adoption Playbook: A Guide to Adopting DevOps in a Multi-Speed IT Enterprise.* Indianapolis, Indiana: John Wiley & Sons, Inc.

BIBLIOGRAFÍA

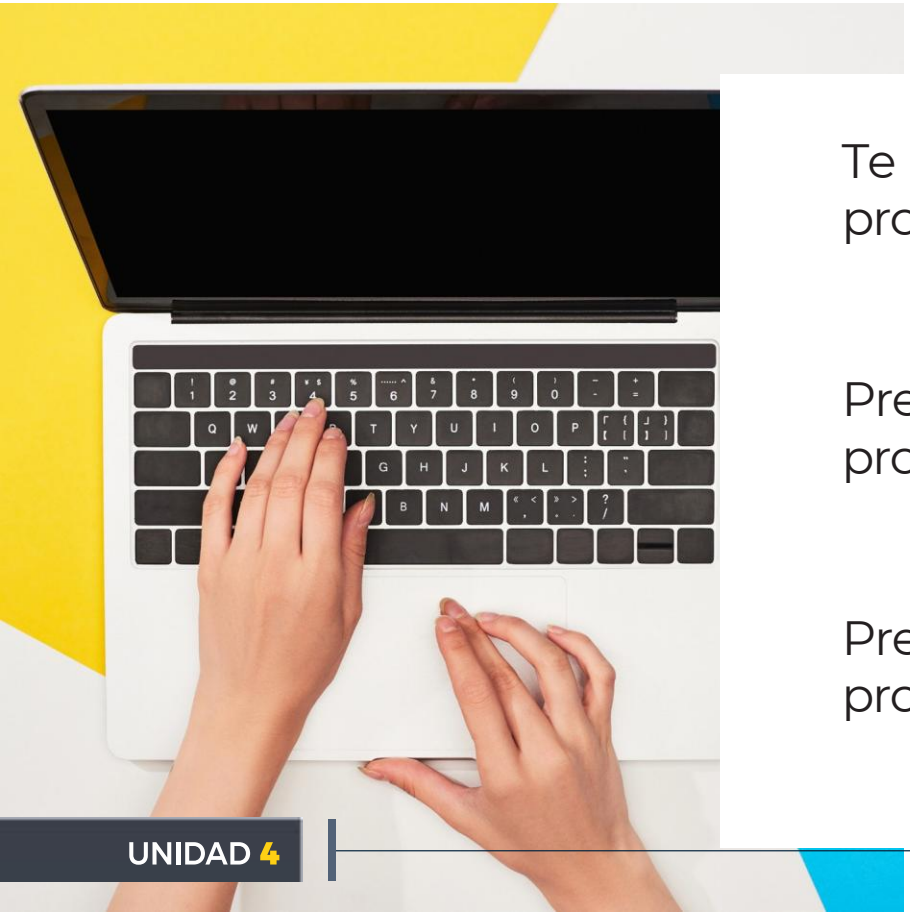


- Vijayakumar, T. (2017). *Practical Azure Application Development*. Colombo, Sri Lanka: Apress.



PROYECTO FINAL

PROYECTO FINAL



Te invitamos a realizar el siguiente proyecto final:

Presiona el botón para descargar el proyecto final:

Presiona el botón para entregar el proyecto final: